
ARM/ARM64 Linux Kernel Memory Management

Complete Reference Guide

A comprehensive technical reference covering all aspects of Linux kernel memory management on ARM 32-bit (ARMv7) and ARM64 (AArch64) architectures — from virtual address space layout and page faults through the buddy allocator, SLUB, CMA, DMA coherency, SMMU, and full MMU translation pipelines including boot-time page table initialization.

[ARM64 / AArch64](#)[ARMv7 / ARM32](#)[Linux Kernel](#)[Memory Management](#)[MMU & TLB](#)[DMA Coherency](#)[SMMU / IOMMU](#)[Buddy Allocator](#)[SLUB Allocator](#)[CMA](#)

Topics: 14 major sections • Includes code examples, ASCII diagrams, assembly, register details, and reference tables

Table of Contents

1 Kernel Virtual Address Space Layout

ARM32 3G/1G Split • ARM64 48-bit VA Split • TTBR registers

2 Why Kernel Space Is Mapped Into Every Process

Syscall performance • Interrupt handling • KPTI post-Meltdown • Permission bits

3 Page Faults — Mechanism and Types

Demand paging • Copy-on-Write • ARM64 ESR/FAR registers • do_page_fault() flow

4 Low Memory and High Memory

LOWMEM direct map • HIGHMEM • kmap/kmap_atomic • ARM64: no highmem

5 Memory Zones

ZONE_DMA • ZONE_NORMAL • ZONE_HIGHMEM • GFP flags • Allocator zone usage

6 Memory Allocation Mechanisms

Buddy allocator • SLUB/kmalloc • vmalloc • DMA allocators • Per-CPU • mempool • Decision guide

7 Buddy Allocator Internals

Orders • free_area lists • Splitting algorithm • Coalescing/merging • Buddy XOR calculation

8 SLUB Allocator Internals

Per-CPU fast path • Partial lists • Freelists storage • NUMA awareness • Debugging • Constructors

9 Memory Compaction and CMA

Fragmentation • Migration types • Two-scanner algorithm • CMA device tree setup • Driver API

10 DMA Coherency and Cache Management on ARM64

CPU cache vs DMA • HW coherent • Cache maintenance instructions • SMMU • Memory types • IOMMU

11 MMU: Virtual-to-Physical Translation — ARM64

4-level page tables • VA breakdown • PTE fields • TLB • ASIDs • Huge pages • Fault types

12 MMU: Virtual-to-Physical Translation — ARM32

2-level page tables • Section mappings • AP/TEX/Domain bits • DACR • TTBCR

13 ARM32 Boot: Initial Kernel Page Table Setup

head.S walkthrough • __create_page_tables • Identity mapping • __enable_mmu • paging_init()

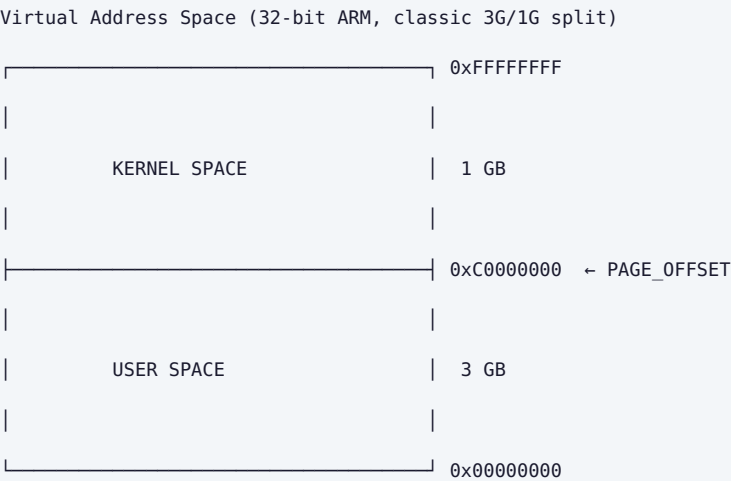
14 ARM32 vs ARM64 Comparison & Reference Tables

SECTION 1

Kernel Virtual Address Space Layout

The virtual address space layout is the foundation of Linux kernel memory management. On ARM32, the CPU has only 4 GB of virtual address space, split between user and kernel. On ARM64, each half gets 128 TB, enabling direct mapping of all physical RAM.

ARM32 (ARMv7) — 3G/1G Split



The kernel uses a single page table per process. The top 1 GB (kernel entries) are **shared across all processes** via the global PGD.

- `PAGE_OFFSET = 0xC0000000` (configurable, e.g., 2G/2G split possible)
- User space: `0x00000000 – 0xBFFFFFFF` (3 GB)
- Kernel space: `0xC0000000 – 0xFFFFFFFF` (1 GB)

ARM64 (AArch64) — 48-bit VA Split

Virtual Address Space (48-bit, VA_BITS=48):

```

0xFFFF_FFFF_FFFF_FFFF 7
                        | Kernel Space (TTBR1_EL1) – 128 TB
0xFFFF_0000_0000_0000 J
                        |
[HOLE – invalid addresses]
(bits 63:48 must all be 0 or all 1)

0x0000_FFFF_FFFF_FFFF 7
                        | User Space (TTBR0_EL1) – 128 TB
0x0000_0000_0000_0000 J

```

- **TTBR0_EL1** → User space page tables (VA bits[63:48] = all 0)
- **TTBR1_EL1** → Kernel space page tables (VA bits[63:48] = all 1)
- **TCR_EL1** → Configures VA size, page size (4KB/16KB/64KB granule), cache attributes

```

// Linux kernel defines:

#define PAGE_OFFSET  0xFFFF800000000000UL // Start of kernel linear map

#define TASK_SIZE    0x0000FFFFFFFFF0000UL // Max user space address

```

ARM64 Kernel Virtual Address Layout

```

0xFFFFFFFFFFFFFFFF 7
                        | vmalloc/ioremap area
0xFFFFF00000000000 |
                        | Linear map of all physical RAM (direct mapping)
0xFFFF800000000000 J ← PAGE_OFFSET

```

On ARM64, all physical RAM is directly mapped starting at `PAGE_OFFSET`:

- Physical 0x0 → Virtual 0xFFFF800000000000
- Physical 0x1000 → Virtual 0xFFFF800000001000
- All pages usable directly — no highmem, no kmap() needed

TCR_EL1 — Translation Control Register

TCR_EL1 key fields:

T0SZ [5:0]: Size offset for TTBR0 (user space)

VA size = $2^{(64 - T0SZ)}$

T0SZ=16 → 48-bit user VA space

T1SZ [21:16]: Size offset for TTBR1 (kernel space)

T1SZ=16 → 48-bit kernel VA space

TG0 [15:14]: Translation Granule for TTBR0

00 = 4KB (most common on Linux ARM64)

01 = 64KB

10 = 16KB (Apple Silicon)

TG1 [31:30]: Translation Granule for TTBR1

SH0/SH1: Shareability (Inner Shareable for normal memory)

IRGN/ORGN: Cache attributes for page table walks

SECTION 2

Why Kernel Space Is Mapped Into Every Process

Why User Process Space Is Shared with Kernel Space

The short answer: **it isn't fully shared — it's *mapped* into the same virtual address space, but with strict access controls.**

The Virtual Address Space Layout

On a typical 32-bit or 64-bit system, each process has its own virtual address space. This space is split into two regions:

- **User space** — where the process's code, stack, heap, and libraries live
- **Kernel space** — where the kernel's code, data structures, and drivers are mapped

On a **32-bit ARM** system (classic split):

- `0x00000000 – 0xBFFFFFFF` → User space (3 GB)
- `0xC0000000 – 0xFFFFFFFF` → Kernel space (1 GB)

On **ARM64 (AArch64)**:

- Lower virtual addresses (e.g., `0x0000...` range) → User space
- Upper virtual addresses (e.g., `0xFFFF...` range) → Kernel space
- This is enforced via **TTBR0_EL1** (user page tables) and **TTBR1_EL1** (kernel page tables)

Why Is Kernel Mapped Into Every Process's Address Space?

Efficient System Calls (Syscall Performance)

When a user process makes a syscall, the CPU switches to kernel mode (EL1 on ARM64). If the kernel were in a completely separate address space, the CPU would need to perform a full **address space switch** (TLB flush, page table swap), which is very expensive. By keeping the kernel mapped in the same virtual address space, the transition is faster — only the privilege level changes.

Interrupt and Exception Handling

Hardware interrupts can occur at any time during user process execution. The CPU must immediately jump to kernel exception handlers. Having the kernel already mapped means no address space switch is needed during interrupt entry.

Memory-Mapped I/O and Driver Access

Kernel drivers need to access hardware registers. These mappings are maintained in kernel space and are always present regardless of which user process is running.

Why Can't User Space Access Kernel Space?

Even though kernel space is *mapped* into the process's virtual address space, **page table entries for kernel pages are marked with privilege bits** (e.g., `PXN`, `UXN`, `AP` bits on ARM). The MMU enforces that:

- User-mode code (EL0) **cannot read or execute** kernel pages

- Any attempt triggers a **permission fault** → segfault or kernel panic

KPTI (Kernel Page Table Isolation) — Post-Meltdown

After the **Meltdown vulnerability (2018)**, Linux introduced **KPTI**:

- In user mode, only a *minimal stub* of the kernel is mapped (just enough to handle syscall entry)
- The full kernel mapping is only present in kernel mode
- This trades some syscall performance for security

On **ARM64**, this is implemented via **KASLR + KPTI** using the `TTBR1_EL1` switch mechanism.

Summary

Aspect	Reason
Kernel mapped in user process space	Fast syscall/interrupt handling without full address space switch
User cannot access kernel pages	MMU enforces privilege-level page protections (AP/PXN bits on ARM)
KPTI splits the mapping further	Mitigates speculative execution side-channel attacks (Meltdown)

Given your ARM64 kernel work, the key registers to study are **TTBR0_EL1**, **TTBR1_EL1**, and the **TCR_EL1** configuration that defines the address space split. Would you like to dive deeper into any of these — such as ARM64 page table walk, KPTI implementation, or how syscall entry/exit manages the address space transition?

SECTION 3

Page Faults — Mechanism, Types, and ARM64 Specifics

A **page fault** is an exception raised by the CPU's Memory Management Unit (MMU) when a running process accesses a virtual address that either:

- Is **not currently mapped** in the page table, or
- Is mapped but the access **violates permissions** (e.g., writing to a read-only page)

It is one of the most fundamental mechanisms in modern OS memory management.

Why Is a Page Fault Needed?

Page faults are not just errors — they are a **deliberate and essential mechanism** that enables:

Demand Paging — Pages are only loaded into physical RAM when actually accessed, not all at once at process startup. This saves memory.

Lazy Allocation — `malloc()` reserves virtual address space but doesn't allocate physical pages until you actually write to them (Copy-on-Write, zero pages).

Memory-Mapped Files — Files are mapped into virtual address space; actual disk reads happen only when the page is accessed.

Swapping / Paging — Pages evicted to disk are marked "not present"; accessing them triggers a page fault that brings them back.

Copy-on-Write (CoW) — After `fork()`, parent and child share the same physical pages (marked read-only). A write triggers a page fault → kernel copies the page → both get their own copy.

Guard Pages / Stack Growth — Stack grows dynamically; a fault below the current stack pointer triggers the kernel to extend the stack.

Types of Page Faults

Type	Cause	Kernel Action
Minor (soft) fault	Page not in TLB but is in RAM	Update TLB/page table, no disk I/O
Major (hard) fault	Page not in RAM (on disk/swap)	Load page from disk into RAM
Invalid fault	Access to unmapped or protected address	Send <code>SIGSEGV</code> to process

How a Page Fault Is Performed — Step by Step

Setup (Before the Fault)

- The OS maintains a **page table** per process mapping virtual → physical addresses
- Each Page Table Entry (PTE) has a **Present bit (P)**:

-
- `P=1` → page is in RAM, access is allowed
 - `P=0` → page is not in RAM (triggers fault)

Fault Sequence

User Process

```

|
|  accesses virtual address 0x4000 (not mapped yet)

```

▼

MMU checks Page Table

```

|
|  Present bit = 0  → CPU raises Page Fault Exception

```

▼

CPU saves registers + PC onto kernel stack

CPU switches to EL1 (kernel mode on ARM64)

|

▼

Kernel Page Fault Handler (do_page_fault on Linux)

|

└─ Is the address in a valid VMA (Virtual Memory Area)?

| NO → Send SIGSEGV → process killed

| YES → Continue

|

└─ Is it a permission violation (write to read-only)?

| YES + CoW → Allocate new page, copy content, update PTE

| YES + no CoW → Send SIGSEGV

|

└─ Is the page in swap?

| YES → Read page from swap device into RAM (major fault)

|

└─ Is it a new anonymous page (e.g., heap/stack)?

| YES → Allocate a zeroed physical page

|

└─ Update Page Table Entry (PTE) → set Present bit = 1

|

▼

Flush TLB entry for that virtual address

|

▼

Return from exception → re-execute the faulting instruction

ARM64 Specifics (Relevant to Your Work)

On **ARM64**, page faults are handled as part of the **synchronous exception** mechanism:

- The CPU vectors to `el1_sync` or `el0_sync` in the exception vector table
- **ESR_EL1** (Exception Syndrome Register) contains the fault reason:
 - `EC = 0b100100` → Data Abort from EL0 (user space)
 - `EC = 0b100101` → Data Abort from EL1 (kernel space)
 - `DFSC` (Data Fault Status Code) tells you: translation fault, permission fault, alignment fault, etc.
- **FAR_EL1** (Fault Address Register) holds the **virtual address that caused the fault**
- Linux ARM64 handler: `do_mem_abort()` → `do_page_fault()`

Concrete Example — Demand Paging

```
#include <stdlib.h>
#include <string.h>

int main() {
    char *buf = malloc(4096); // (1) Virtual address allocated, NO physical page yet
    buf = 'A';                // (2) PAGE FAULT triggered here!
    return 0;
}
```

What happens at step (2):

CPU tries to write to `buf` → MMU checks PTE → Present bit = 0

CPU raises a **Data Abort** exception → jumps to kernel exception handler

Kernel's `do_page_fault()` runs:

- Checks: is `buf` in a valid VMA? → Yes (heap region)
- Allocates a **zeroed physical page** (4 KB)
- Updates the PTE: maps virtual `buf` → new physical page, sets `P=1`, `RW=1`
- Flushes TLB

Returns to user space → **re-executes** `buf = 'A'` → succeeds this time

The process never knew a fault happened — it's completely transparent.

Copy-on-Write Example (fork)

```
int x = 10;

pid_t pid = fork();

if (pid == 0) {
    x = 20;    // PAGE FAULT here in child! CoW triggered
}
```

After `fork()`, parent and child share the same physical page for `x`, marked **read-only**. When the child writes `x = 20`:

- Page fault → permission fault (write to read-only)
- Kernel detects CoW → allocates new page, copies content
- Child's PTE updated to point to new page, marked writable
- Parent's page unchanged

Summary

Aspect	Detail
Trigger	MMU raises exception on invalid/missing virtual address access
ARM64 registers	<code>ESR_EL1</code> (fault reason), <code>FAR_EL1</code> (fault address), <code>ELR_EL1</code> (return address)
Key Linux function	<code>do_page_fault()</code> → <code>handle_mm_fault()</code>
Transparent to user	Yes — faulting instruction is re-executed after resolution
Performance cost	Minor fault: microseconds; Major fault (disk I/O): milliseconds

Given your ARM64 kernel work, a great next step would be studying `arch/arm64/mm/fault.c` in the Linux kernel source — it's where `do_mem_abort()` dispatches page faults. Would you like to go deeper into **TLB management**, **VMA internals**, or **how swap works with page faults**?

SECTION 4

Low Memory and High Memory

Low memory and high memory is one of the most important — and often confusing — topics in Linux kernel memory management, especially relevant to your ARM/ARM64 and Qualcomm SoC work. Let me give you the complete picture.

Background: Why This Problem Exists

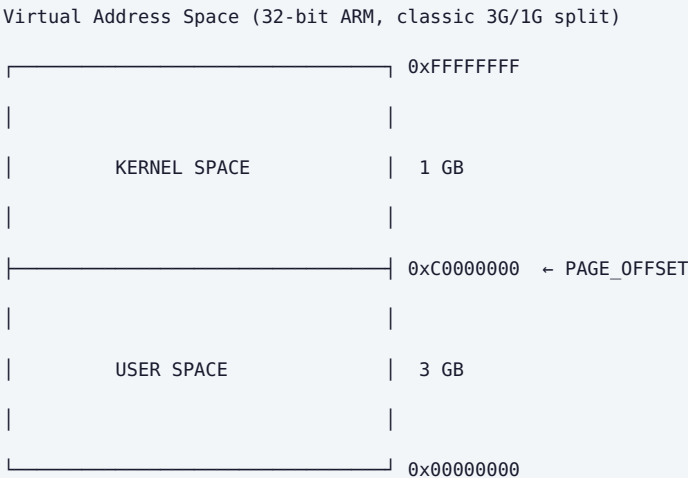
This problem is **specific to 32-bit systems** (ARM, x86). On **64-bit (ARM64)**, the virtual address space is so large (48-bit or 52-bit) that this problem essentially disappears. But understanding it is critical because:

- Many embedded/IoT SoCs still run 32-bit kernels
- The concepts explain kernel memory architecture fundamentals
- Legacy code and drivers still reference these concepts

The Core Problem on 32-bit ARM

On a 32-bit system, the CPU can address only **4 GB of virtual address space** (2^{32}).

The kernel splits this virtual address space between user and kernel:



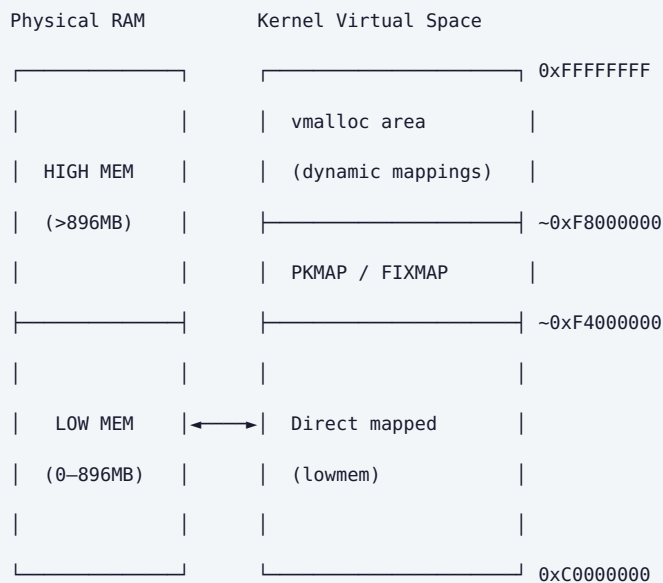
The kernel has only **1 GB of virtual address space** to work with.

Now suppose the machine has **2 GB or more of physical RAM**. The kernel **cannot directly map all of physical RAM** into its 1 GB virtual window. This creates the split:

Low Memory (LOWMEM)

Definition: Physical memory that is **permanently and directly mapped** into the kernel's virtual address space.

- On ARM with `PAGE_OFFSET = 0xC0000000`, the kernel maps physical RAM starting at `0xC0000000` virtual
- The mapping is: `virtual = physical + PAGE_OFFSET`
- This region is called **lowmem** or the **direct-mapped region**
- Typically covers the first **~896 MB** of physical RAM (on x86; ARM varies by config)
- Kernel can access lowmem addresses **directly** — no mapping setup needed
- `__pa(vaddr)` / `__va(paddr)` macros do the conversion instantly



Key properties of lowmem:

- Always mapped — kernel can dereference a lowmem pointer anytime
- Used for: kernel code, kernel data, page tables, slab/slub allocator, most kernel allocations
- `kmalloc()` allocates from lowmem (physically contiguous, directly mapped)
- `alloc_pages()` for low memory returns directly usable pages

High Memory (HIGHMEM)

Definition: Physical memory that **cannot be permanently mapped** into the kernel's virtual address space due to address space exhaustion.

- Physical RAM above ~896 MB (the exact boundary is `ZONE_HIGHMEM` start)
- The kernel **knows this memory exists** but has no permanent virtual address for it
- To use a highmem page, the kernel must **temporarily map** it into a small reserved virtual window, use it, then **unmap** it

Three Mechanisms to Access Highmem

1. `kmap()` / `kunmap()` — Permanent-ish mapping (sleepable)

```
struct page *page = alloc_page(GFP_HIGHUSER);

void *vaddr = kmap(page);      // Maps page into PKMAP area

memcpy(vaddr, src, PAGE_SIZE); // Use it

kunmap(page);                 // Unmap when done
```

- Uses the **PKMAP** (Persistent Kernel Map) area: a small window (~2–4 MB) of virtual addresses
- Limited slots available — can **sleep/block** if all slots are in use
- Safe to use in process context

2. `kmap_atomic()` / `kunmap_atomic()` — Temporary atomic mapping

```
void *vaddr = kmap_atomic(page); // Uses FIXMAP slots

// CRITICAL: Cannot sleep, cannot be preempted here

memcpy(vaddr, src, PAGE_SIZE);

kunmap_atomic(vaddr);
```

- Uses **FIXMAP** area: fixed virtual addresses reserved at compile time
- Very fast — no locking
- **Cannot sleep** between `kmap_atomic` and `kunmap_atomic`
- Used in interrupt handlers, softirqs, atomic contexts
- On ARM64 and modern kernels, `kmap_atomic` is essentially a no-op (highmem doesn't exist)

3. **bounce buffers** — For DMA

- DMA controllers often can't access highmem (limited address bus)
- Kernel allocates a lowmem "bounce buffer", copies data to/from highmem page
- Transparent to the driver

Memory Zones

The kernel divides physical memory into **zones** to manage these constraints:

```
ZONE_DMA      — Low physical addresses for legacy DMA (e.g., <16MB on x86)
ZONE_NORMAL   — Directly mapped lowmem (the main workhorse)
ZONE_HIGHMEM  — High physical memory, not permanently mapped (32-bit only)
```

On ARM 32-bit:

```

ZONE_DMA      → 0 – some low boundary (platform-specific)
ZONE_NORMAL   → up to ~768MB or 896MB physical
ZONE_HIGHMEM  → everything above that

```

On ARM64:

```

ZONE_DMA      → low addresses (for 32-bit DMA-capable devices)
ZONE_NORMAL   → everything else (no HIGHMEM zone at all)

```

Allocator Behavior with Zones

Allocator	Zone Used	Notes
<code>kmalloc(size, GFP_KERNEL)</code>	ZONE_NORMAL (lowmem)	Physically contiguous, directly mapped
<code>kmalloc(size, GFP_DMA)</code>	ZONE_DMA	For DMA-capable buffers
<code>vmalloc(size)</code>	Any zone (virtually contiguous)	Not physically contiguous
<code>alloc_page(GFP_HIGHUSER)</code>	ZONE_HIGHMEM preferred	For user pages, needs kmap to access
<code>get_free_page(GFP_KERNEL)</code>	ZONE_NORMAL	Single page, directly usable

GFP Flags — What a Developer Must Know

GFP = **Get Free Pages** flags. They control which zone and behavior:

Flag	Meaning
<code>GFP_KERNEL</code>	Normal kernel allocation, can sleep, uses ZONE_NORMAL
<code>GFP_ATOMIC</code>	Cannot sleep (interrupt context), may fail
<code>GFP_DMA</code>	Must come from ZONE_DMA
<code>GFP_HIGHUSER</code>	Allocate from highmem for user pages
<code>GFP_NOWAIT</code>	Don't wait/sleep, return NULL if unavailable
<code>_GFP_ZERO</code>	Zero the allocated page
<code>_GFP_HIGHMEM</code>	Allow allocation from ZONE_HIGHMEM

Rule of thumb:

- In **process context** (can sleep): use `GFP_KERNEL`

- In **interrupt/atomic context** (cannot sleep): use `GFP_ATOMIC`
- For **DMA buffers**: use `GFP_DMA` or `dma_alloc_coherent()`

vmalloc vs kmalloc — The Highmem Connection

`kmalloc`:

- Physically contiguous
- Directly mapped (lowmem)
- Fast (no TLB setup)
- Limited by contiguous free lowmem
- Max ~4MB typically

`vmalloc`:

- Virtually contiguous, physically scattered
- Uses `vmalloc` area in kernel virtual space
- Can use highmem pages
- Slower (TLB entries per page)
- Good for large allocations (modules, firmware buffers)

When to use which:

- `kmalloc` → DMA, hardware registers, small/medium kernel structures
- `vmalloc` → Large buffers, kernel modules, non-DMA use

ARM64: Highmem Is Gone

On **ARM64 (AArch64)**, the virtual address space is 48-bit (256 TB per side):

- Kernel gets `0xFFFF000000000000 – 0xFFFFFFFFFFFFFFFF` (128 TB)
- **All physical RAM is directly mapped** — no highmem zone
- `kmap()` / `kunmap()` are no-ops (just return the linear map address)
- `CONFIG_HIGHMEM` is not set on ARM64

The **linear map** on ARM64:

```
Physical 0x0      → Virtual 0xFFFF800000000000 (PAGE_OFFSET)
Physical 0x1000   → Virtual 0xFFFF800000001000
...all RAM mapped...
```

This is why ARM64 drivers are simpler — you never need `kmap`.

Practical Developer Rules

1. Never dereference a `struct page *` directly

```
// WRONG

void *ptr = (void *)page; // page is a metadata struct, not the data!

// RIGHT (lowmem page)

void *ptr = page_address(page); // Returns NULL for unmapped highmem

// RIGHT (highmem safe)

void *ptr = kmap(page);

// ... use ptr ...

kunmap(page);
```

2. `page_address()` returns NULL for highmem pages

```
struct page *p = alloc_page(GFP_HIGHUSER);

void *va = page_address(p); // NULL if highmem!

// Must use kmap() instead
```

3. DMA allocations must come from lowmem

```
// Correct DMA allocation

dma_addr_t dma_handle;

void *cpu_addr = dma_alloc_coherent(dev, size, &dma_handle, GFP_KERNEL);

// This ensures the buffer is in DMA-accessible memory
```

4. Highmem pages in interrupt context

```
// In interrupt handler – use kmap_atomic
irqreturn_t my_irq(int irq, void *dev) {
    struct page *page = get_my_page();

    void *vaddr = kmap_atomic(page);

    process_data(vaddr);

    kunmap_atomic(vaddr);

    return IRQ_HANDLED;
}
```

Key Kernel Macros and Functions

Macro/Function	Purpose
<code>__pa(vaddr)</code>	Virtual → Physical (lowmem only)
<code>__va(paddr)</code>	Physical → Virtual (lowmem only)
<code>page_address(page)</code>	Get virtual address of page (NULL if highmem)
<code>kmap(page)</code>	Map highmem page (sleepable)
<code>kunmap(page)</code>	Unmap highmem page
<code>kmap_atomic(page)</code>	Map highmem page (atomic, no sleep)
<code>kunmap_atomic(vaddr)</code>	Unmap atomic mapping
<code>PageHighMem(page)</code>	Check if page is in highmem zone
<code>virt_to_page(vaddr)</code>	Get <code>struct page</code> from virtual address
<code>pfn_to_page(pfn)</code>	Get <code>struct page</code> from page frame number

Summary Table

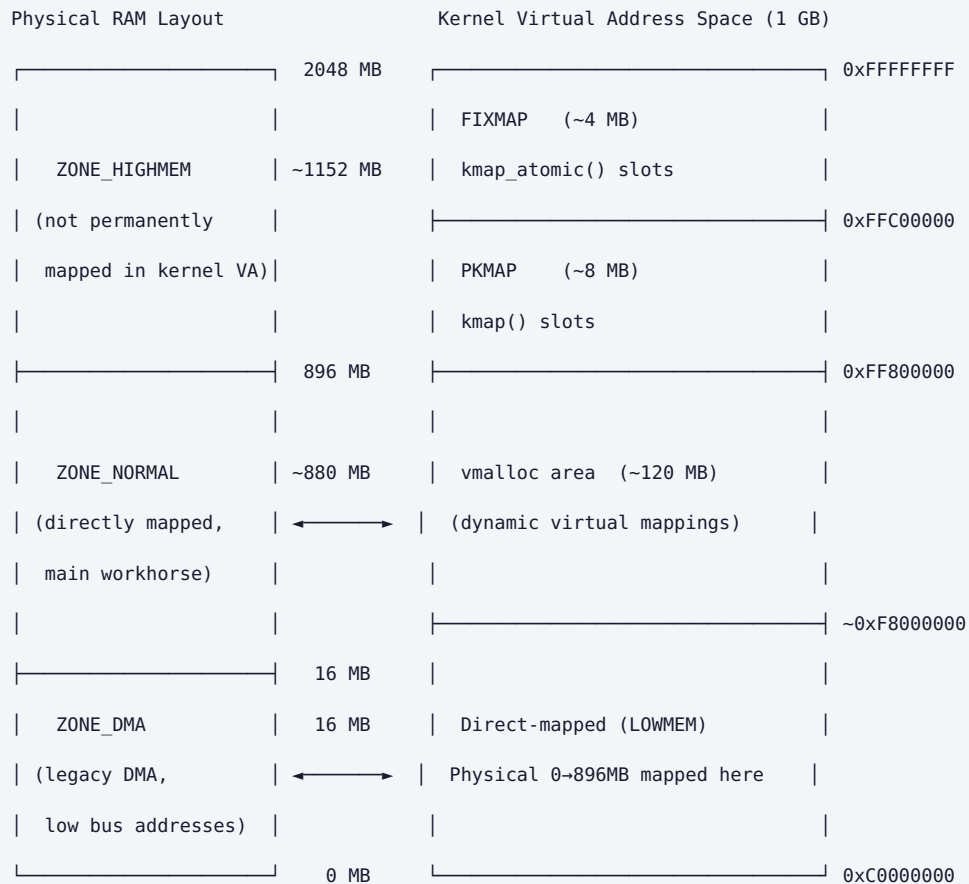
Aspect	Low Memory	High Memory
Physical range	0 – ~896 MB	> ~896 MB
Kernel virtual mapping	Permanent, direct	Temporary, on demand
Access method	Direct pointer dereference	<code>kmap()</code> / <code>kmap_atomic()</code>
<code>page_address()</code>	Returns valid pointer	Returns NULL
DMA capable	Yes	Usually No
ARM64	All memory is lowmem	Does not exist
Zone	<code>ZONE_NORMAL</code> / <code>ZONE_DMA</code>	<code>ZONE_HIGHMEM</code>

Given your ARM64 and Qualcomm SoC background, you'll mostly deal with the **linear map** and `dma_alloc_coherent()` patterns. But for any 32-bit ARM legacy code or driver porting work, `kmap` / `kunmap` patterns are essential to recognize. Would you like to go deeper into **ZONE internals and the buddy allocator**, **slab/slub allocator**, or **DMA coherency and cache management on ARM**?

SECTION 5

Memory Zones

Memory Zones on 32-bit ARM



ARM64 Memory Zones

On ARM64 (AArch64), there is **no ZONE_HIGHMEM** — the 48-bit virtual address space is large enough to directly map all physical RAM:

ARM64 Zone Layout:

ZONE_DMA → low physical addresses (32-bit DMA-capable devices)

ZONE NORMAL → everything else (no HIGHMEM)

Zone Allocator Behavior

Allocator	Zone Used	Notes
<code>kmalloc(size, GFP_KERNEL)</code>	ZONE_NORMAL	Physically contiguous, directly mapped
<code>kmalloc(size, GFP_DMA)</code>	ZONE_DMA	For DMA-capable buffers
<code>vmalloc(size)</code>	Any zone	Virtually contiguous only
<code>alloc_page(GFP_HIGHUSER)</code>	ZONE_HIGHMEM preferred	Needs <code>kmap()</code> to access
<code>get_free_page(GFP_KERNEL)</code>	ZONE_NORMAL	Single page, directly usable

GFP Flags — Critical Reference

GFP = **Get Free Pages** flags, controlling which zone and allocation behavior:

Flag	Meaning
<code>GFP_KERNEL</code>	Normal kernel allocation, can sleep, uses ZONE_NORMAL
<code>GFP_ATOMIC</code>	Cannot sleep (interrupt context), may fail
<code>GFP_DMA</code>	Must come from ZONE_DMA
<code>GFP_HIGHUSER</code>	Allocate from highmem for user pages
<code>GFP_NOWAIT</code>	Don't wait/sleep, return NULL if unavailable
<code>__GFP_ZERO</code>	Zero the allocated page
<code>__GFP_HIGHMEM</code>	Allow allocation from ZONE_HIGHMEM

Rule of thumb:

- In **process context** (can sleep): use `GFP_KERNEL`
- In **interrupt/atomic context** (cannot sleep): use `GFP_ATOMIC`
- For **DMA buffers**: use `GFP_DMA` or `dma_alloc_coherent()`

SECTION 6

Memory Allocation Mechanisms

Linux kernel memory allocation is a rich topic — and given your ARM64/Qualcomm SoC and driver development background, mastering this is essential. Here's the complete picture: **what each allocator is, why it exists, where to use it, and how to use it correctly.**

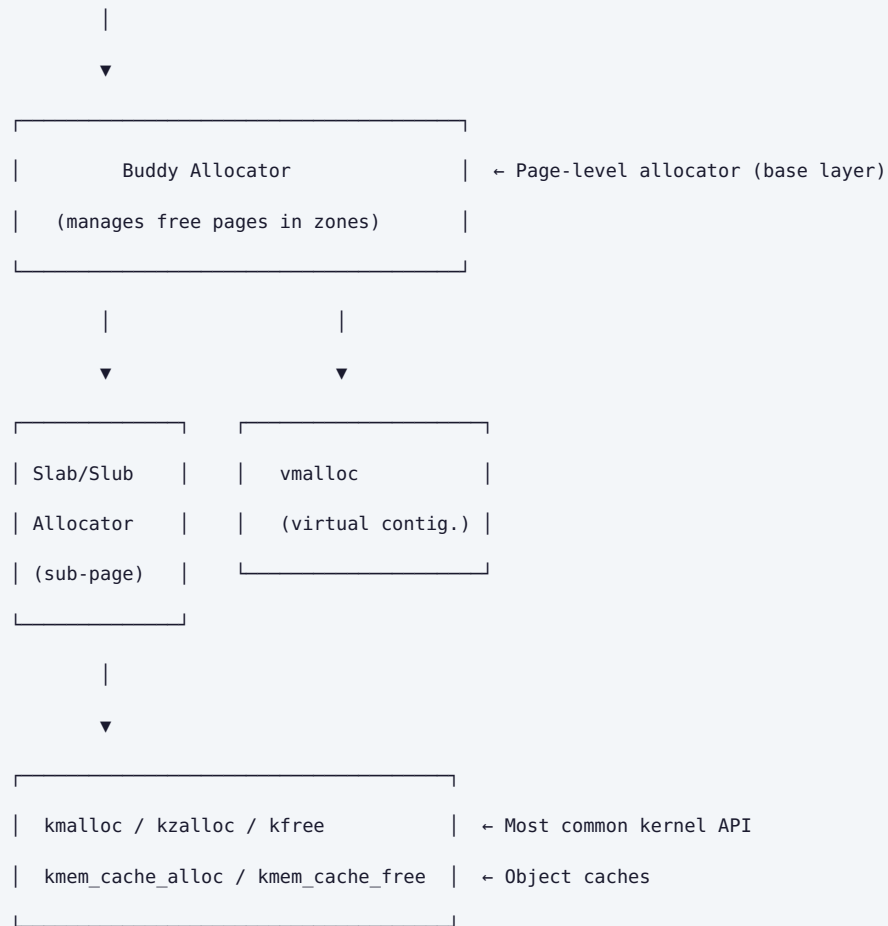
| The Big Picture — Why Multiple Allocators?

No single allocator fits all needs because kernel memory has wildly different requirements:

- Some allocations need to be **physically contiguous** (DMA)
- Some need to be **fast** (interrupt handlers)
- Some are **large** (firmware buffers, module loading)
- Some are **tiny and frequent** (kernel objects, inodes)
- Some need **special alignment** (hardware registers)

The kernel provides a layered system:

Hardware Physical RAM



1. Buddy Allocator — The Foundation

What It Is

The **buddy allocator** is the lowest-level page allocator. It manages **free physical pages** in power-of-2 blocks (order 0 = 1 page, order 1 = 2 pages, ..., order 11 = 2048 pages on most systems).

How It Works

- Maintains free lists for each order: `free_area` through `free_area[MAX_ORDER]`
- When you request N pages, it finds the smallest order $\geq N$, splits if needed
- When freeing, it **merges ("coalesces") buddies** back into larger blocks
- This prevents fragmentation over time

API

```
// Allocate 2^order contiguous pages

struct page *alloc_pages(gfp_t gfp_mask, unsigned int order);

struct page *alloc_page(gfp_t gfp_mask);           // order 0 = 1 page


// Get virtual address directly

unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order);

unsigned long __get_free_page(gfp_t gfp_mask);     // 1 page


// Free

__free_pages(struct page *page, unsigned int order);

free_pages(unsigned long addr, unsigned int order);

free_page(unsigned long addr);
```

When to Use

- When you need **multiple physically contiguous pages**
- When you need a `struct page *` (e.g., for DMA mapping, page cache)
- Rarely used directly in drivers — prefer `kmalloc` or `dma_alloc_coherent`

Example

```
// Allocate 4 contiguous pages (order=2, 2^2=4 pages = 16KB)

struct page *pages = alloc_pages(GFP_KERNEL, 2);

if (!pages)

    return -ENOMEM;


void *vaddr = page_address(pages); // Get virtual address

// ... use vaddr ...

__free_pages(pages, 2);
```

2. Slab / Slub Allocator — Sub-Page Object Allocator

What It Is

The **slab allocator** (modern implementation: **SLUB**) sits on top of the buddy allocator and provides efficient allocation of **small, fixed-size objects**. It solves the problem of internal fragmentation when you need many small allocations.

Why It Exists

If you allocate a 64-byte structure using the buddy allocator, you waste 4032 bytes of a 4096-byte page. The slab allocator:

- Groups many same-sized objects into a single page
- Maintains **per-CPU caches** for hot objects (very fast, no locking)
- Reuses freed objects without returning to the buddy allocator
- Reduces cache misses by keeping objects in CPU cache

Two Usage Modes

Mode 1: General-purpose caches via `kmalloc`

The kernel pre-creates caches for common sizes: 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192 bytes.

```
// Allocate
void *ptr = kmalloc(size, gfp_flags);
void *ptr = kzalloc(size, gfp_flags); // Zero-initialized (preferred)
void *ptr = kcalloc(n, size, gfp_flags); // Array allocation, zero-initialized

// Reallocate
void *ptr = krealloc(ptr, new_size, gfp_flags);

// Free
kfree(ptr);
kfree_sensitive(ptr); // Zeroes memory before freeing (security-sensitive data)
```

Mode 2: Dedicated object caches via `kmem_cache`

For frequently allocated structs (e.g., `struct inode`, `struct task_struct`):

```
// Create a cache (typically at module init)
struct kmem_cache *my_cache = kmem_cache_create(
    "my_object_cache",    // name (visible in /proc/slabinfo)
    sizeof(struct my_obj), // object size
    0,                    // alignment (0 = default)
    SLAB_HWCACHE_ALIGN,  // flags
    NULL                  // constructor (optional)
);

// Allocate an object
struct my_obj *obj = kmem_cache_alloc(my_cache, GFP_KERNEL);

// Free an object
kmem_cache_free(my_cache, obj);

// Destroy cache (at module exit)
kmem_cache_destroy(my_cache);
```

kmalloc Size Limits

- Practical max: **~4 MB** (order 10 pages), but physically contiguous
- For anything > a few KB, consider `vmalloc` instead
- Check `/proc/slabinfo` or `/sys/kernel/slab/` for cache statistics

GFP Flags — Critical for Correct Usage

Flag	Context	Behavior
<code>GFP_KERNEL</code>	Process context	Can sleep, reclaim memory, best effort
<code>GFP_ATOMIC</code>	Interrupt / atomic context	Cannot sleep, may fail, uses emergency reserves
<code>GFP_NOWAIT</code>	Process context	Cannot sleep, no reclaim
<code>GFP_NOIO</code>	I/O path (avoid deadlock)	Can sleep but no I/O
<code>GFP_NOFS</code>	Filesystem path	Can sleep but no filesystem ops
<code>GFP_DMA</code>	DMA buffers	Allocates from ZONE_DMA
<code>__GFP_ZERO</code>	Any	Zero the memory
<code>__GFP_NOFAIL</code>	Critical paths	Loops until success (use sparingly)

The most important rule:

```
// In process context (can sleep):
ptr = kmalloc(size, GFP_KERNEL);

// In interrupt handler / spinlock held / atomic context:
ptr = kmalloc(size, GFP_ATOMIC);

// NEVER use GFP_KERNEL while holding a spinlock – deadlock risk!
```

3. vmalloc — Large Virtual-Contiguous Allocations

What It Is

`vmalloc` allocates memory that is **virtually contiguous** but **not necessarily physically contiguous**. It uses the `vmalloc` area of kernel virtual space.

Why It Exists

- `kmalloc` requires physically contiguous pages — hard to find for large sizes
- `vmalloc` can stitch together scattered physical pages into a contiguous virtual range
- Ideal for large allocations where physical contiguity is not required

API

```
void *vmalloc(unsigned long size);
void *vzalloc(unsigned long size);    // Zero-initialized
void *vmalloc_node(unsigned long size, int node); // NUMA-aware
void vfree(const void *addr);
```

When to Use vs. kmalloc

Criterion	kmalloc	vmalloc
Physical contiguity	Required	Not required
DMA capable	Yes	No (pages may not be contiguous)
Size limit	~4 MB	Limited by virtual address space
Speed	Fast (no TLB setup)	Slower (per-page TLB entries)
Use case	Drivers, DMA, small structs	Modules, firmware, large buffers

Example — Loading Firmware

```
// Firmware buffer – large, no DMA needed
void *fw_buf = vmalloc(firmware_size);
if (!fw_buf)
    return -ENOMEM;

memcpy(fw_buf, firmware_data, firmware_size);
// process firmware...
vfree(fw_buf);
```

Example — Kernel Module

Kernel modules themselves are loaded into vmalloc space:

```
/proc/vmallocinfo ← shows all vmalloc allocations
```

4. DMA Allocators — For Hardware Interaction

This is critical for your Qualcomm SoC and driver work.

The Problem

DMA engines have limited address bus width and cache coherency requirements. You need:

- Physically contiguous memory
- Known physical (bus) address
- Cache-coherent access (or explicit cache management)

`dma_alloc_coherent` — The Standard DMA API

```
// Allocate coherent DMA memory

void *cpu_addr = dma_alloc_coherent(
    dev,          // struct device * (your platform/PCI device)
    size,         // bytes
    &dma_handle,  // output: DMA (bus) address for hardware
    GFP_KERNEL
);

// Use:
// cpu_addr → CPU virtual address (for kernel to read/write)
// dma_handle → physical/bus address (program into DMA controller registers)

// Free
dma_free_coherent(dev, size, cpu_addr, dma_handle);
```

Coherent means: CPU and DMA engine always see the same data — no explicit cache flush needed. On ARM64, this typically means the memory is mapped as **non-cacheable** or uses hardware cache coherency.

`dma_alloc_writecombine` / `dma_alloc_wc`

```
// Write-combining: CPU writes are batched, good for framebuffers

void *cpu_addr = dma_alloc_wc(dev, size, &dma_handle, GFP_KERNEL);
```

Streaming DMA (for existing buffers)

When you have an existing buffer and want to DMA to/from it:

```
// Map for DMA (flushes cache)
dma_addr_t dma_addr = dma_map_single(
    dev,
    cpu_ptr,          // kernel virtual address
    size,
    DMA_TO_DEVICE     // or DMA_FROM_DEVICE, DMA_BIDIRECTIONAL
);

if (dma_mapping_error(dev, dma_addr))
    return -ENOMEM;

// Program dma_addr into hardware...
// After DMA completes:
dma_unmap_single(dev, dma_addr, size, DMA_TO_DEVICE);
// Now safe to access cpu_ptr again
```

DMA Pools — For Small Repeated DMA Allocations

```
// Create pool (e.g., for descriptor rings)
struct dma_pool *pool = dma_pool_create(
    "my_desc_pool",
    dev,
    sizeof(struct my_descriptor), // object size
    32,                          // alignment
    0                            // boundary (0 = none)
);

// Allocate from pool
dma_addr_t dma_addr;
struct my_descriptor *desc = dma_pool_alloc(pool, GFP_KERNEL, &dma_addr);

// Free
dma_pool_free(pool, desc, dma_addr);

// Destroy pool
dma_pool_destroy(pool);
```

5. Per-CPU Allocations

For data that is **per-CPU** (no locking needed, cache-friendly):

```
// Static per-CPU variable
DEFINE_PER_CPU(int, my_counter);

// Access (must disable preemption)
int val = get_cpu_var(my_counter);
put_cpu_var(my_counter);

// Or with explicit CPU
per_cpu(my_counter, cpu_id) = 42;

// Dynamic per-CPU allocation
int __percpu *ptr = alloc_percpu(int);
*per_cpu_ptr(ptr, smp_processor_id()) = 42;
free_percpu(ptr);
```

Use case: Performance counters, per-CPU queues, lock-free statistics.

6. mempool — Guaranteed Allocation Pool

For situations where allocation **must not fail** (e.g., I/O path, memory pressure):

```
// Create pool with 16 pre-allocated objects
mempool_t *pool = mempool_create_kmalloc_pool(16, sizeof(struct my_obj));

// Allocate (will block if pool empty, but never fails if can_wait)
struct my_obj *obj = mempool_alloc(pool, GFP_NOIO);

// Free (returns to pool or releases to slab)
mempool_free(obj, pool);

mempool_destroy(pool);
```

Use case: Block device drivers, SCSI stack — anywhere you can't afford allocation failure during I/O.

7. Stack Memory — `alloca` / Local Variables

The kernel stack is **small and fixed** (typically 8 KB or 16 KB per thread on ARM64):

```
// OK – small local variables
void my_func(void) {
    int buf;           // Fine
    struct small_obj obj; // Fine
}

// DANGEROUS – large stack allocations
void bad_func(void) {
    char buf[4096];     // Stack overflow risk!
    // Use kmalloc instead
}
```

Rule: Never allocate more than a few hundred bytes on the kernel stack. Use `kmalloc` for anything larger.

Complete Decision Guide

Need memory in kernel?

|

└─ Is it for DMA / hardware access?

|

└─ Small, repeated (descriptors) → `dma_pool_alloc()`

|

└─ Large buffer → `dma_alloc_coherent()`

|

└─ Is it physically contiguous (non-DMA)?

|

└─ Small (< ~4MB) → `kmalloc()` / `kzalloc()`

|

└─ Multiple pages → `alloc_pages()`

|

└─ Does physical contiguity matter?

|

└─ No, large buffer → `vmalloc()` / `vzalloc()`

|

└─ Is it a frequently allocated fixed-size object?

|

└─ Yes → `kmem_cache_create()` + `kmem_cache_alloc()`

|

└─ Is it per-CPU data?

|

└─ Yes → `alloc_percpu()` / `DEFINE_PER_CPU()`

|

└─ Must allocation never fail (I/O path)?

|

└─ Yes → `mempool_alloc()`

|

└─ Small, temporary, in process context?

└─ Stack variable (< ~256 bytes)

Common Mistakes to Avoid

1. Using `GFP_KERNEL` in atomic context

```
spin_lock(&my_lock);  
ptr = kmalloc(size, GFP_KERNEL); // BUG: can sleep while holding spinlock!  
spin_unlock(&my_lock);  
  
// Fix:  
spin_lock(&my_lock);  
ptr = kmalloc(size, GFP_ATOMIC); // Correct  
spin_unlock(&my_lock);
```

2. Not checking allocation return value

```
// WRONG  
ptr = kmalloc(size, GFP_KERNEL);  
ptr->field = value; // NULL dereference if allocation failed!  
  
// RIGHT  
ptr = kmalloc(size, GFP_KERNEL);  
if (!ptr)  
    return -ENOMEM;  
ptr->field = value;
```

3. Using `vmalloc` for DMA

```
// WRONG – vmalloc pages are not physically contiguous  
buf = vmalloc(size);  
dma_handle = virt_to_phys(buf); // WRONG! Not valid for vmalloc  
  
// RIGHT  
buf = dma_alloc_coherent(dev, size, &dma_handle, GFP_KERNEL);
```

4. Memory leaks in error paths

```
int my_probe(struct platform_device *pdev) {
    priv->buf = kmalloc(BUF_SIZE, GFP_KERNEL);
    if (!priv->buf)
        return -ENOMEM;

    priv->irq = request_irq(...);
    if (priv->irq < 0) {
        kfree(priv->buf);    // Must free on error!
        return priv->irq;
    }

    return 0;
}
```

5. Double free

```
kfree(ptr);
kfree(ptr); // BUG: double free, heap corruption

// Fix: set to NULL after free
kfree(ptr);
ptr = NULL;
```

Debugging Memory Issues

Tool	Purpose
<code>KASAN</code> (Kernel Address Sanitizer)	Detects use-after-free, out-of-bounds
<code>KMEMLEAK</code>	Detects memory leaks
<code>SLUB_DEBUG</code>	Slab corruption detection
<code>/proc/slabinfo</code>	Slab cache statistics
<code>/proc/vmallocinfo</code>	vmalloc allocations
<code>/proc/buddyinfo</code>	Buddy allocator free lists per zone
<code>kmem_cache_alloc_trace</code>	Allocation tracing
<code>valgrind</code> (userspace equivalent)	Not available in kernel; use KASAN instead

Enable in kernel config:

```
CONFIG_KASAN=y
CONFIG_KMEMLEAK=y
CONFIG_DEBUG_SLAB=y
CONFIG_DEBUG_PAGEALLOC=y
```

Summary Reference Table

Allocator	Physically Contiguous	DMA Safe	Can Sleep	Size Limit	Use Case
<code>kmalloc</code>	Yes	Yes	Depends on GFP	~4 MB	General kernel objects
<code>kzalloc</code>	Yes	Yes	Depends on GFP	~4 MB	Same, zero-initialized
<code>vmalloc</code>	No	No	Yes	Virtual space	Large non-DMA buffers
<code>alloc_pages</code>	Yes	Yes	Depends on GFP	Order-based	Page-level control
<code>dma_alloc_coherent</code>	Yes	Yes	Yes	Platform limit	DMA buffers
<code>dma_pool_alloc</code>	Yes	Yes	Yes	Pool size	Small DMA objects
<code>kmem_cache_alloc</code>	Yes	Yes	Depends on GFP	Cache size	Frequent fixed-size objects
<code>mempool_alloc</code>	Yes	Yes	Yes	Pool size	Must-not-fail paths
<code>alloc_percpu</code>	N/A	No	Yes	Unlimited	Per-CPU data

Given your Qualcomm SoC driver work, the most important allocators to master are `kmalloc` / `kzalloc` for general use, `dma_alloc_coherent` and `dma_pool` for hardware interaction, and `kmem_cache` for high-frequency object allocation in drivers like I2C/SPI controllers. Would you like to go deeper into **DMA coherency and cache management on ARM64, SLUB internals**, or see **complete driver examples** using these allocators?

SECTION 7

Buddy Allocator Internals

The Linux **Buddy Allocator** works by maintaining free lists organized by "order" (power-of-2 page blocks) and splitting/merging blocks as needed. Here's a detailed visual walkthrough.

What Is an "Order"?

Each order represents a block of **2ⁿ contiguous pages**:

Order	Pages	Size (4KB pages)
0	1 page	4 KB
1	2 pages	8 KB
2	4 pages	16 KB
3	8 pages	32 KB
4	16 pages	64 KB
...	...	...
10	1024 pages	4 MB
11 (MAX_ORDER)	2048 pages	8 MB

The Free Lists Structure

The buddy allocator maintains an array of free lists — one per order:

```
free_area → [ ]→[ ]→[ ]→[ ]    (free 1-page blocks)
free_area → [ ][ ]→[ ][ ]    (free 2-page blocks)
free_area → [ ][ ][ ][ ]    (free 4-page blocks)
free_area → (empty)
free_area → [ ][ ][ ][ ][ ][ ][ ][ ]    (free 16-page blocks)
...
free_area → (empty)
free_area → [ × 2048 pages ]    (one huge free block)
```

Each entry in the list is a **physically contiguous block** of 2ⁿ pages, aligned to 2ⁿ page boundaries.

Scenario 1: Allocation — Splitting a Block

Request: Allocate 1 page (order 0)

Initial state — only a free order-2 block (4 pages) exists:

BEFORE:

free_area → (empty)

free_area → (empty)

free_area → [P0][P1][P2][P3] ← one free block of 4 pages

Step 1: No order-0 block available → go up to order-1 → still empty

Step 2: No order-1 block → go up to order-2 → FOUND!

Step 3: Split order-2 block into two order-1 blocks:

free_area → (empty)

free_area → [P0][P1] + [P2][P3] ← two order-1 blocks

free_area → (empty)

Step 4: Take one order-1 block, split into two order-0 blocks:

free_area → [P0] + [P1] ← two order-0 blocks

free_area → [P2][P3] ← one order-1 block remains

free_area → (empty)

Step 5: Allocate [P0] → remove from free list, return to caller

AFTER:

free_area → [P1] ← P1 remains free

free_area → [P2][P3] ← P2-P3 remain free

free_area → (empty)

Allocated: [P0] ✓

Visual split tree:

```

[P0][P1][P2][P3]   (order-2 block)
|
├─ [P0][P1]   (order-1 buddy A)
|   |
|   └─ [P0]   ← ALLOCATED (order-0)
|       └─ [P1] ← stays in free_area
|
└─ [P2][P3]   ← stays in free_area

```

Scenario 2: Freeing — Merging (Coalescing) Buddies

Now free [P0] back:

BEFORE freeing P0:

free_area → [P1]

free_area → [P2][P3]

Step 1: P0 is freed → check if its BUDDY (P1) is also free

Buddy of P0 at order-0 = P1 (they are "buddies" because they came from the same split)

Step 2: P1 IS free → merge P0 + P1 → form order-1 block [P0][P1]

Remove P1 from free_area

free_area → (empty)

free_area → [P0][P1] + [P2][P3]

Step 3: Check if buddy of [P0][P1] at order-1 is free

Buddy = [P2][P3] → IS free!

Step 4: Merge [P0][P1] + [P2][P3] → form order-2 block [P0][P1][P2][P3]

Remove [P2][P3] from free_area

AFTER:

free_area → (empty)

free_area → (empty)

free_area → [P0][P1][P2][P3] ← fully restored!

Visual merge tree:

[P0] freed

+ buddy [P1] is free

→ merge → [P0][P1] (order-1)

+ buddy [P2][P3] is free

→ merge → [P0][P1][P2][P3] (order-2) ✓

How "Buddy" Is Determined

Two blocks are **buddies** if and only if:

They are the **same size** (same order)

They are **adjacent** in physical memory

They came from the **same parent block** (their base addresses differ by exactly 2^n pages)

The buddy address is calculated with a simple XOR:

```
// Given page frame number (pfn) and order:
buddy_pfn = pfn ^ (1 << order);

// Example: pfn=0, order=0 → buddy = 0 ^ 1 = 1 (P1 is buddy of P0)
// Example: pfn=0, order=1 → buddy = 0 ^ 2 = 2 (P2-P3 is buddy of P0-P1)
```

This is why it's called the **buddy** system — each block has exactly one buddy at each order level.

Full State Diagram — Allocation and Free Sequence

Physical Pages: [P0] [P1] [P2] [P3] [P4] [P5] [P6] [P7]

↑	↑
0	7

Initial State (all free, one order-3 block):

free_area → [P0..P7]

ACTION: Allocate 1 page (order-0)

Split order-3 → two order-2:

free_area → [P0..P3] [P4..P7]

Split [P0..P3] order-2 → two order-1:

free_area → [P0,P1] [P2,P3]

free_area → [P4..P7]

Split [P0,P1] order-1 → two order-0:

free_area → [P0] [P1]

free_area → [P2,P3]

free_area → [P4..P7]

Allocate [P0]:

free_area → [P1]

free_area → [P2,P3]

free_area → [P4..P7]

ACTION: Allocate 4 pages (order-2)

order-2 block [P4..P7] is available → allocate directly:

free_area → [P1]

free_area → [P2,P3]

```
free_area → (empty) ← [P4..P7] allocated
```

```
ACTION: Free [P0]
```

```
P0 freed → buddy P1 is free → merge → [P0,P1]
```

```
[P0,P1] → buddy [P2,P3] is free → merge → [P0..P3]
```

```
[P0..P3] → buddy [P4..P7] is ALLOCATED → stop
```

```
free_area → (empty)
```

```
free_area → (empty)
```

```
free_area → [P0..P3] ← merged back
```

```
ACTION: Free [P4..P7]
```

```
[P4..P7] freed → buddy [P0..P3] is free → merge → [P0..P7]
```

```
free_area → [P0..P7] ← fully restored to initial state! ✓
```

Why This Prevents Fragmentation

Without coalescing, repeated alloc/free cycles would leave memory like this:

```
[ALLOC][FREE][ALLOC][FREE][ALLOC][FREE][ALLOC][FREE]
```

Even though 50% of memory is free, you can't satisfy a 2-page request because no two adjacent free pages exist — this is **external fragmentation**.

The buddy system solves this by **aggressively merging** free blocks whenever possible, ensuring large contiguous blocks are reconstructed as soon as their buddies are freed.

Linux Kernel Source Reference

For your ARM64 kernel work, the relevant files are:

- `mm/page_alloc.c` — `__alloc_pages()`, `__free_pages()`, `expand()` (splitting), `__free_one_page()` (merging)
- `include/linux/mmzone.h` — `struct free_area`, `struct zone`
- `mm/internal.h` — buddy PFN calculation macros

Key functions to study:

```
__alloc_pages_nodemask()    // top-level allocator
get_page_from_freelist()    // finds a free block
expand()                   // splits a block down to requested order
__free_one_page()          // frees and coalesces buddies
find_buddy_pfn()           // calculates buddy address (XOR trick)
```

Would you like to continue with the **SLUB allocator internals** (how it sits on top of the buddy allocator), or move to **memory compaction and migration** (how the kernel deals with long-term fragmentation that even the buddy system can't prevent)?

SECTION 8

SLUB Allocator Internals

The **SLUB allocator** is the default slab allocator in the Linux kernel (since ~2.6.23), replacing the original SLAB and the simpler SLOB. It manages sub-page memory allocations efficiently by grouping same-sized objects into **slabs** (pages or groups of pages), with heavy optimization for multi-core systems.

Why SLUB Exists — The Problem It Solves

The buddy allocator works at **page granularity** (minimum 4 KB). But the kernel constantly allocates tiny objects:

- `struct task_struct` (~7 KB on ARM64)
- `struct inode` (~600 bytes)
- `struct sk_buff` (~240 bytes)
- `kmalloc(64)` calls from drivers

Without a sub-page allocator, every 64-byte allocation would waste 4032 bytes. SLUB solves this by **packing many objects into a single page**.

Core Concepts

The `kmem_cache` — The Heart of SLUB

Every type of object has a `kmem_cache` (also called a "cache" or "slab cache"):

```
struct kmem_cache {
    struct kmem_cache_cpu __percpu *cpu_slab; // Per-CPU fast path
    unsigned int size; // Object size (with metadata)
    unsigned int object_size; // Actual object size
    unsigned int offset; // Free pointer offset within object
    unsigned int min_partial; // Min partial slabs to keep
    int refcount;
    struct list_head list; // All caches linked here
    struct kmem_cache_node *node[MAX_NUMNODES]; // Per-NUMA-node lists
    // ...
};
```

The kernel pre-creates caches for `kmalloc` sizes (8, 16, 32, 64, 128, ... bytes) and drivers create their own with `kmem_cache_create()`.

SLUB Architecture — Three Levels

```

kmem_cache
|
|
├─ cpu_slab (per-CPU)           ← FAST PATH (no locking)
|   └─ freelist                 ← pointer to next free object on current page
|   └─ page *                   ← the active slab page being used
|
|
├─ node[N] (per-NUMA node)     ← SLOW PATH (needs locking)
|   └─ partial list             ← pages with some free + some used objects
|   └─ (full pages tracked implicitly via page flags)

```

Level 1: Per-CPU Slab (Fastest Path)

Each CPU has its own **active slab page** and a **freelist** — a linked list of free objects within that page. Allocation from the per-CPU slab requires **no locking** at all:

Per-CPU slab page (e.g., 4KB, object size = 64 bytes → 63 objects per page)

```
|
| [obj0] [obj1] [obj2] [obj3] ... [obj62]
|
| ↑
|
| freelist → obj0 → obj3 → obj7 → obj15 → NULL
|
| (free objects form a singly-linked list via
|
|   a pointer stored inside the free object itself)
|
```

Allocation (fast path):

```
// Pseudocode of what SLUB does for kcalloc(64, GFP_KERNEL):
object = cpu_slab->freelist;           // grab head of freelist
cpu_slab->freelist = object->next;      // advance freelist
return object;                          // done - no lock needed!
```

This is **extremely fast** — essentially just two pointer operations.

Level 2: Partial List (Node Level)

When the per-CPU slab page is **full** (no free objects), SLUB:

- Moves the full page off the per-CPU slot

- Grabs a **partial page** from the node's partial list (pages with some free objects)

- Makes it the new per-CPU active slab

```
node->partial list:
    [page: 20/63 free] → [page: 5/63 free] → [page: 45/63 free] → NULL
```

Level 3: New Page from Buddy Allocator

When the partial list is also empty, SLUB requests a **new page** from the buddy allocator (`alloc_pages()`), initializes all objects as free, and makes it the active per-CPU slab.

The Free Pointer — How the Freelist Works Inside Objects

SLUB stores the **next free object pointer** *inside* the free object itself (at a fixed offset). This is elegant — no separate metadata array needed:

```
Free object layout (object_size = 64 bytes, offset = 0):

Bytes 0-7:  [ ptr to next free object ]  ← used by SLUB when object is FREE
Bytes 8-63: [ ... unused ... ]

Allocated object layout:

Bytes 0-63: [ user data ]                ← SLUB doesn't touch this
```

When an object is allocated, the user owns all bytes. When freed, SLUB writes the next-free pointer into the first 8 bytes (or at `offset` if the object has a destructor/constructor that needs byte 0 preserved).

Full Allocation Flow — Step by Step

```
kmalloc(64, GFP_KERNEL)
|
├─ [FAST PATH] Check per-CPU freelist
|   ├─ freelist != NULL → pop object, return ✓ (no lock, ~ns)
|   └─ freelist == NULL → slow path
|
├─ [SLOW PATH] Refill per-CPU slab
|   ├─ Lock node's partial list
|   ├─ Partial page available?
|   |   ├─ YES → move to per-CPU, unlock, retry fast path
|   |   └─ NO → allocate new page from buddy allocator
|   |       initialize all objects as free
|   |       set as per-CPU active slab
|   |       unlock, retry fast path
|   └─ Buddy allocator failed → return NULL (OOM)
|
└─ Return allocated object pointer
```

Full Free Flow — Step by Step

```
kfree(ptr)
|
|─ Find the slab page: page = virt_to_head_page(ptr)
|
|
|─ [FAST PATH] Does ptr belong to THIS CPU's active slab?
|   |─ YES → push ptr onto per-CPU freelist ✓ (no lock)
|   |─ NO → slow path
|
|
|─ [SLOW PATH] ptr belongs to a different CPU's slab or partial slab
|   |─ Lock the page's freelist
|   |─ Add ptr to page's freelist
|   |─ Was page full? → move to node partial list
|   |─ Is page now completely empty?
|       |─ partial list has > min_partial pages → return page to buddy allocator
|       |─ else → keep in partial list (avoid thrashing)
|   |─ Unlock
```

SLUB vs SLAB vs SLOB

Feature	SLOB	SLAB	SLUB
Target	Tiny embedded systems	General purpose (old)	General purpose (modern default)
Complexity	Minimal	High (coloring, per-CPU queues)	Medium
Per-CPU optimization	No	Yes (complex)	Yes (simpler, better)
NUMA awareness	No	Yes	Yes
Debugging support	Minimal	Good	Excellent
Memory overhead	Lowest	High	Low
Fragmentation	High	Low	Low
Default since	Never (opt-in)	Pre-2.6.23	2.6.23+

SLUB Debugging — Built-in Poison and Red Zones

SLUB has excellent built-in debugging (enabled with `CONFIG_SLUB_DEBUG=y`):

Poisoning

Free objects are filled with `0x6b` ("kk" pattern). If you see `0x6b6b6b6b` in a crash dump, you're reading a freed object — **use-after-free** bug.

Allocated objects are filled with `0x5a` before being handed to the caller (with `SLAB_POISON` flag).

```
Free object:  [6b 6b 6b 6b 6b 6b 6b 6b ...]
Allocated:    [5a 5a 5a 5a 5a 5a 5a 5a ...] (before user writes)
```

Red Zones

SLUB adds **red zone bytes** before and after each object (with `SLAB_RED_ZONE`). If these are overwritten, SLUB detects a **buffer overflow**:

```
[RED_ZONE][ object data ][RED_ZONE][free_ptr]
0xbb...  user's bytes  0xbb...
```

Free Pointer Validation

With `CONFIG_SLAB_FREELIST_HARDENED=y`, the free pointer is **XOR-encoded** with a random secret:

```
// Stored as: ptr XOR secret XOR address_of_slot
// Prevents attackers from overwriting freelist pointers to redirect allocations
```

SLUB and NUMA

On NUMA systems (multiple memory nodes), SLUB maintains **per-node partial lists**:

```
kmem_cache
├─ cpu_slab[CPU0] → active page on Node 0
├─ cpu_slab[CPU1] → active page on Node 0
├─ cpu_slab[CPU2] → active page on Node 1
├─ node->partial → pages from Node 0 memory
└─ node->partial → pages from Node 1 memory
```

Allocations prefer memory local to the current CPU's NUMA node, reducing cross-node memory access latency.

Object Constructor / Destructor

`kmem_cache_create()` accepts an optional **constructor**:

```
void my_ctor(void *obj) {
    struct my_obj *p = obj;

    mutex_init(&p->lock);        // Initialize once, reused across alloc/free

    INIT_LIST_HEAD(&p->list);
}

struct kmem_cache *cache = kmem_cache_create(
    "my_cache",
    sizeof(struct my_obj),
    0,
    SLAB_HWCACHE_ALIGN,
    my_ctor           // called once when object is first created
);
```

The constructor is called **once per object** when the slab page is first populated — not on every `kmem_cache_alloc()`. This means objects retain their initialized state between alloc/free cycles (e.g., mutexes stay initialized). This is a significant performance win for objects with expensive initialization.

Important `kmem_cache_create` Flags

Flag	Effect
<code>SLAB_HWCACHE_ALIGN</code>	Align objects to CPU cache line (prevents false sharing)
<code>SLAB_POISON</code>	Fill with poison pattern on free (debug)
<code>SLAB_RED_ZONE</code>	Add red zones around objects (overflow detection)
<code>SLAB_PANIC</code>	Panic if cache creation fails (for critical caches)
<code>SLAB_ACCOUNT</code>	Account memory to kmemcg (cgroup memory tracking)
<code>SLAB_TYPESAFE_BY_RCU</code>	Safe for RCU-protected lookups (don't free page while RCU readers exist)
<code>SLAB_RECLAIM_ACCOUNT</code>	Objects are reclaimable (e.g., inode cache)

Viewing SLUB State at Runtime

```
# All slab caches – name, active objects, total objects, object size
```

```
cat /proc/slabinfo
```

```
# Detailed per-cache info
```

```
cat /sys/kernel/slab/<cache_name>/alloc_calls
```

```
cat /sys/kernel/slab/<cache_name>/free_calls
```

```
cat /sys/kernel/slab/<cache_name>/objects
```

```
cat /sys/kernel/slab/<cache_name>/slabs
```

```
# Summary of memory used by slabs
```

```
slabtop
```

```
# Validate all slab caches (triggers SLUB debug checks)
```

```
echo 1 > /sys/kernel/slab/<cache_name>/validate
```

Complete SLUB Lifecycle Diagram

```
kmem_cache_create("my_cache", 64, ...)
```

```
|
| Creates kmem_cache struct
| Registers with /sys/kernel/slab/my_cache
|
```

▼

```
kmem_cache_alloc(cache, GFP_KERNEL) ← First call
```

```
|
| Per-CPU freelist empty
| Node partial list empty
| → alloc_pages(GFP_KERNEL, order) from buddy allocator
| → Initialize page: link all 63 objects as free chain
| → Set as per-CPU active slab
| → Pop first object from freelist
|
```

▼

```
[Object returned to caller]
```

```
|
| ... caller uses object ...
|
```

▼

```
kmem_cache_free(cache, obj)
```

```
|
| obj belongs to per-CPU active slab?
| YES → push onto per-CPU freelist (no lock)
|
```

▼

```
kmem_cache_alloc(cache, GFP_KERNEL) ← Second call
```

```
|
| Per-CPU freelist has obj → pop and return immediately
| (Object reused – constructor NOT called again)
|
```

▼

```
kmem_cache_destroy(cache) ← Module exit
```

```
|
```

- | All objects must be freed first (warns if not)
- | Returns all slab pages to buddy allocator
- | Unregisters from /sys/kernel/slab/

SLUB in Your Driver Work (ARM64 / Qualcomm SoC)

For your driver development, the most common patterns are:

Pattern 1: Per-device private data

```
static struct kmem_cache *my_dev_cache;

static int __init my_driver_init(void) {
    my_dev_cache = kmem_cache_create("my_dev",
        sizeof(struct my_device_priv),
        0, SLAB_HWCACHE_ALIGN, NULL);
    return my_dev_cache ? 0 : -ENOMEM;
}

static int my_probe(struct platform_device *pdev) {
    struct my_device_priv *priv = kmem_cache_alloc(my_dev_cache, GFP_KERNEL);
    if (!priv)
        return -ENOMEM;
    platform_set_drvdata(pdev, priv);
    // ...
}

static int my_remove(struct platform_device *pdev) {
    struct my_device_priv *priv = platform_get_drvdata(pdev);
    kmem_cache_free(my_dev_cache, priv);
    return 0;
}

static void __exit my_driver_exit(void) {
    kmem_cache_destroy(my_dev_cache);
}
```

Pattern 2: High-frequency small allocations (e.g., I2C message descriptors)

```
// Instead of kcalloc() in the hot path:

struct kmem_cache *msg_cache = kmem_cache_create("i2c_msg",
    sizeof(struct i2c_msg), 0, SLAB_HWCACHE_ALIGN, NULL);

// In interrupt/transfer path:

struct i2c_msg *msg = kmem_cache_alloc(msg_cache, GFP_ATOMIC);

// ... fill and submit msg ...

kmem_cache_free(msg_cache, msg);
```

Summary

Aspect	Detail
Fast path	Per-CPU freelist — no locking, pointer pop/push
Slow path	Node partial list → new page from buddy allocator
Freelist storage	Inside free objects themselves (at <code>offset</code>)
NUMA aware	Per-node partial lists
Debugging	Poison (0x6b/0x5a), red zones, freelist hardening
Constructor	Called once per object creation, not per alloc
Key files	<code>mm/slub.c</code> , <code>include/linux/slab.h</code>
Runtime info	<code>/proc/slabinfo</code> , <code>/sys/kernel/slab/</code> , <code>slabtop</code>

The SLUB allocator is a masterpiece of lock-free design for the common case — the per-CPU fast path makes `kmalloc` / `kfree` nearly as fast as stack allocation for small objects. Given your ARM64 driver work, understanding the **per-CPU freelist** and `SLAB_HWCACHE_ALIGN` flag will directly impact your driver's performance.

Would you like to continue with **memory compaction and the CMA (Contiguous Memory Allocator)** — which is critical for your Qualcomm SoC DMA work — or dive into **slab shrinkers and memory pressure handling**?

SECTION 9

Memory Compaction and CMA

Continuing from the SLUB allocator, here's a deep dive into **Memory Compaction** and the **CMA (Contiguous Memory Allocator)** — both critically important for your Qualcomm SoC and DMA-heavy driver work.

The Problem: Fragmentation Over Time

Even with the buddy allocator's coalescing, long-running systems suffer from **external fragmentation**:

Physical Memory after hours of operation:

[USED] [FREE] [USED] [FREE] [USED] [FREE] [USED] [FREE] [USED] [FREE]

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9

Free pages: P1, P3, P5, P7, P9 (5 pages free = 20 KB)

But largest contiguous free block: 1 page (4 KB)

→ Cannot satisfy a 2-page (8 KB) contiguous allocation!

→ Even though 20 KB is free in total

This is the fragmentation problem. Two mechanisms address it:

Memory Compaction — moves pages around to create contiguous free regions

CMA — reserves a region upfront that is guaranteed to be contiguous

Part 1: Memory Compaction

What It Is

Memory compaction **migrates movable pages** to consolidate free space into large contiguous blocks. It's like defragmenting a hard drive, but for RAM.

Key Insight: Not All Pages Are Movable

The kernel classifies pages into **migration types**:

Migration Type	Description	Examples
<code>MIGRATE_UNMOVABLE</code>	Cannot be moved	Kernel code, kmalloc buffers, DMA buffers
<code>MIGRATE_MOVABLE</code>	Can be moved (page table updated)	User process pages, page cache
<code>MIGRATE_RECLAIMABLE</code>	Can be dropped and reloaded	File-backed pages, inode cache
<code>MIGRATE_CMA</code>	Reserved for CMA allocations	CMA region pages
<code>MIGRATE_ISOLATE</code>	Temporarily isolated during migration	Pages being migrated right now

The buddy allocator maintains **separate free lists per migration type** within each order. This is called **anti-fragmentation** — it tries to keep unmovable pages clustered together so movable pages can be compacted freely.

How Compaction Works

BEFORE compaction:

Physical Memory:

[UNMOV][MOV][UNMOV][MOV][FREE][MOV][UNMOV][FREE][MOV][FREE]

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9

Goal: Create a large contiguous free block

Step 1: Scanner A starts from LOW end → finds movable pages (P1, P3, P5, P8)

Step 2: Scanner B starts from HIGH end → finds free pages (P4, P7, P9)

Step 3: Migrate movable pages into free slots at high end:

P1 → P9 (copy content, update page table)

P3 → P7 (copy content, update page table)

Step 4: P1 and P3 are now free

AFTER compaction:

[UNMOV][FREE][UNMOV][FREE][FREE][MOV][UNMOV][FREE][MOV][MOV]

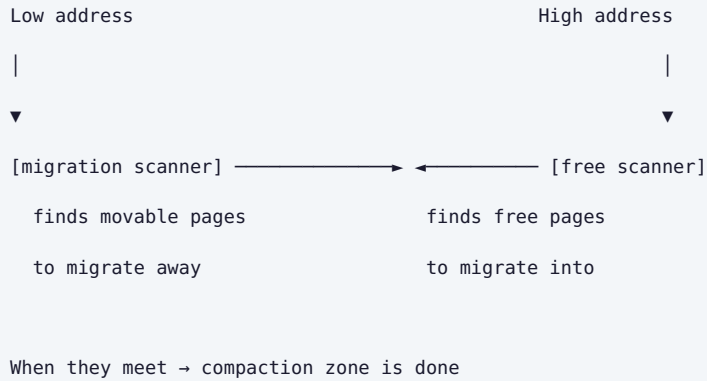
P0 P1 P2 P3 P4 P5 P6 P7 P8 P9

Free contiguous block: P1, P3, P4 → not quite...

(In practice, compaction runs until a sufficiently large free block is formed)

The Two Scanners

Linux compaction uses two concurrent scanners that meet in the middle:



Compaction Triggers

Compaction is triggered:

Automatically — when a high-order allocation fails and memory pressure is moderate

Proactively — `kcompactd` daemon runs in background (one per NUMA node)

Manually — `echo 1 > /proc/sys/vm/compact_memory`

Via `khugepaged` — when trying to collapse pages into a huge page (THP)

Compaction Tuning

```

# How aggressively to compact (0=disabled, 1=light, 2=moderate, 3=aggressive)
/proc/sys/vm/compaction_proactiveness # 0-100, default 20

# Watermark scale factor
/proc/sys/vm/watermark_scale_factor

# Check compaction statistics
cat /proc/vmstat | grep compact

# compact_migrate_scanned
# compact_free_scanned
# compact_isolated
# compact_stall      ← allocation stalled waiting for compaction
# compact_success
# compact_fail

```

Compaction Cost

- **CPU cost:** Scanning pages, copying data, updating page tables, TLB flushes

-
- **Latency:** Can cause **allocation stalls** (tens to hundreds of milliseconds)
 - **Not always possible:** Unmovable pages act as "anchors" that block compaction
-

| Part 2: CMA — Contiguous Memory Allocator

What Problem CMA Solves

Some hardware **requires** large physically contiguous buffers at runtime:

- **Camera ISP** — 4K video frame buffers (tens of MB)
- **GPU** — command buffers, texture memory
- **Video codec** — decode/encode buffers
- **DSP** — firmware + data buffers

These can't be satisfied by compaction alone because:

- Compaction is slow and not guaranteed to succeed
- The allocation may be needed at interrupt time (can't wait)
- The buffer may be very large (hundreds of MB)

CMA's solution: Reserve a region of physical memory at boot time. During normal operation, the kernel uses this region for **movable pages** (so it's not wasted). When a driver needs a large contiguous buffer, CMA **evicts the movable pages** (migrates them elsewhere) and hands the now-contiguous region to the driver.

CMA Architecture

Physical Memory Layout with CMA:



CMA Setup — Device Tree (Your Qualcomm SoC World)

On ARM/ARM64 SoCs, CMA regions are defined in the **Device Tree**:

```

/ {
    reserved-memory {
        #address-cells = <2>;

        #size-cells = <2>;

        ranges;

        /* Default CMA region – used by dma_alloc_coherent */
        linux,cma {
            compatible = "shared-dma-pool";

            reusable;

            size = <0x0 0x10000000>; /* 256 MB */

            alignment = <0x0 0x400000>; /* 4 MB aligned */

            linux,cma-default;
        };

        /* Dedicated CMA for camera ISP */
        camera_cma: camera_region {
            compatible = "shared-dma-pool";

            reusable;

            reg = <0x0 0x90000000 0x0 0x20000000>; /* fixed: 512MB, 512MB size */

            linux,cma-default;
        };

        /* Carveout – NOT reusable, always reserved */
        secure_region: secure@0xA0000000 {
            compatible = "shared-dma-pool";

            no-map; /* not reusable, not in page allocator */

            reg = <0x0 0xA0000000 0x0 0x4000000>; /* 64 MB */
        };
    };

    camera@1234000 {
        memory-region = <&camera_cma>; /* bind device to CMA region */
    };
}

```

```
};
```

Key DT properties:

- `reusable` → CMA: kernel can use pages normally, evicts when driver needs them
- `no-map` → Carveout: completely reserved, never used by kernel, not in buddy allocator
- `linux,cma-default` → this is the default CMA pool for `dma_alloc_coherent()`

CMA Kernel Configuration

```
CONFIG_CMA=y
CONFIG_CMA_SIZE_MBYTES=256      # Default CMA size (if not set in DT)
CONFIG_CMA_ALIGNMENT=8         # Minimum alignment (2^8 = 256 pages)
CONFIG_DMA_CMA=y               # Enable CMA for DMA allocations
CONFIG_CMA_DEBUGFS=y           # /sys/kernel/debug/cma/ stats
```

CMA Allocation API

From Driver Code (High-Level — Preferred)

```
// dma_alloc_coherent automatically uses CMA if available
void *cpu_addr = dma_alloc_coherent(dev, size, &dma_handle, GFP_KERNEL);
// → internally calls cma_alloc() if size > threshold and CMA is configured
// → migrates movable pages out of CMA region
// → returns contiguous physical pages

dma_free_coherent(dev, size, cpu_addr, dma_handle);
// → returns pages to CMA region (marked movable again)
```

Direct CMA API (Low-Level — For Custom Allocators)

```
#include <linux/cma.h>

// Allocate 'count' contiguous pages from a specific CMA area
struct page *pages = cma_alloc(
    cma,          // struct cma * (from of_reserved_mem_device_init())
    count,        // number of pages
    align,        // alignment in pages (power of 2)
    no_warn       // suppress warnings on failure
);

if (!pages)
    return -ENOMEM;

// Get physical address
phys_addr_t phys = page_to_phys(pages);

// Free back to CMA
cma_release(cma, pages, count);
```

Binding a Device to a CMA Region

```
// In driver probe – bind device to its DT-specified memory-region
static int my_probe(struct platform_device *pdev) {
    int ret = of_reserved_mem_device_init(&pdev->dev);

    if (ret) {
        dev_err(&pdev->dev, "Failed to init reserved memory
");
        return ret;
    }

    // Now dma_alloc_coherent() uses the device's CMA region
    priv->buf = dma_alloc_coherent(&pdev->dev, BUF_SIZE,
                                   &priv->dma_addr, GFP_KERNEL);

    // ...
}

static int my_remove(struct platform_device *pdev) {
    dma_free_coherent(&pdev->dev, BUF_SIZE, priv->buf, priv->dma_addr);
    of_reserved_mem_device_release(&pdev->dev);

    return 0;
}
```

CMA Allocation Flow — Internals

```

dma_alloc_coherent(dev, 256MB, &dma_handle, GFP_KERNEL)
|
├─ Size > threshold → try CMA
|
├─ cma_alloc(cma_area, 65536_pages, align, false)
|   |
|   └─ Find a free contiguous range in CMA bitmap
|       (CMA tracks which pages are allocated via a bitmap)
|   |
|   └─ Pages found but occupied by movable user pages
|       |
|       └─ isolate_migratepages_range() ← mark pages as MIGRATE_ISOLATE
|           |
|           └─ migrate_pages() ← copy content to other free pages
|               |
|               └─ Allocate replacement pages outside CMA
|                   |
|                   └─ Copy page content
|                       |
|                       └─ Update page tables (rmap)
|                           |
|                           └─ TLB flush
|                               |
|                               └─ Pages are now free within CMA region
|                                   |
|                                   └─ Mark pages as allocated in CMA bitmap
|                                       |
|                                       └─ Return struct page * (contiguous!)
|                                           |
|                                           └─ Map pages into kernel virtual space (if needed)
|                                               |
|                                               └─ Return cpu_addr + dma_handle to driver

```

CMA vs. Carveout vs. Compaction

Aspect	Compaction	CMA	Carveout (<code>no-map</code>)
Memory reserved at boot	No	Yes (reusable)	Yes (hard reserved)
Normal kernel use of region	N/A	Yes (movable pages)	No
Memory waste	None	None (reused)	Full reservation
Allocation guarantee	Not guaranteed	High (may fail if unmovable pages present)	Always succeeds
Allocation latency	High (scan + migrate)	Medium (migrate CMA pages)	Instant
Use case	General defrag	Large DMA buffers (camera, GPU)	Secure memory, firmware
DT property	N/A	<code>reusable</code>	<code>no-map</code>

Huge Pages and THP — Related Concept

Transparent Huge Pages (THP) use compaction to create **2 MB pages** (on ARM64) instead of 4 KB pages:

Normal pages: [4K][4K][4K]...[4K] (512 pages = 2 MB)

 ↓ khugepaged collapses these ↓

Huge page: [2 MB]

Benefits:

- Fewer TLB entries needed (512x reduction)
- Better performance for large memory workloads
- ARM64: uses 2 MB block entries in page tables (level 2)

THP relies on compaction to find 512 contiguous free 4K pages to form a 2 MB block.

```
# THP settings
cat /sys/kernel/mm/transparent_hugepage/enabled

# [always] madvise never

echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
```

Debugging CMA and Compaction

```
# CMA statistics

cat /sys/kernel/debug/cma/<cma_name>/alloc_count

cat /sys/kernel/debug/cma/<cma_name>/release_count

cat /sys/kernel/debug/cma/<cma_name>/fail_count


# CMA bitmap (which pages are allocated)

cat /sys/kernel/debug/cma/<cma_name>/bitmap


# Compaction statistics

cat /proc/vmstat | grep -E "compact|migrate"


# Memory zone info (shows free pages per order per zone)

cat /proc/buddyinfo


# Detailed zone info including migration type free lists

cat /proc/pagetypeinfo


# Force compaction

echo 1 > /proc/sys/vm/compact_memory


# Check if CMA is configured

cat /proc/meminfo | grep Cma

# CmaTotal:      262144 kB

# CmaFree:       245760 kB
```

Practical Qualcomm SoC Example — Camera Driver

Here's a realistic pattern from a camera ISP driver (similar to what you'd see on Qualcomm platforms):

```

struct cam_isp_dev {
    struct device      *dev;
    void               *frame_buf;
    dma_addr_t         frame_dma;
    size_t             frame_size;
};

static int cam_isp_probe(struct platform_device *pdev) {
    struct cam_isp_dev *isp;

    isp = devm_kzalloc(&pdev->dev, sizeof(*isp), GFP_KERNEL);
    if (!isp)
        return -ENOMEM;

    isp->dev = &pdev->dev;

    /* Bind to camera_cma region defined in DT */
    ret = of_reserved_mem_device_init(&pdev->dev);
    if (ret) {
        dev_err(&pdev->dev, "No reserved memory region
");
        return ret;
    }

    /* Allocate 4K video frame buffer from CMA */
    isp->frame_size = 4096 * 2160 * 4; /* 4K RGBA = ~33 MB */
    isp->frame_buf = dma_alloc_coherent(&pdev->dev,
                                       isp->frame_size,
                                       &isp->frame_dma,
                                       GFP_KERNEL);

    if (!isp->frame_buf) {
        dev_err(&pdev->dev, "Failed to allocate frame buffer
");
        of_reserved_mem_device_release(&pdev->dev);
        return -ENOMEM;
    }
}

```

```
    }

    dev_info(&pdev->dev, "Frame buffer: CPU=%p DMA=%pad size=%zu\n",
            isp->frame_buf, &isp->frame_dma, isp->frame_size);

    /* Program DMA address into ISP hardware registers */
    writel(lower_32_bits(isp->frame_dma), isp->base + ISP_FRAME_ADDR_LO);
    writel(upper_32_bits(isp->frame_dma), isp->base + ISP_FRAME_ADDR_HI);

    platform_set_drvdata(pdev, isp);
    return 0;
}

static int cam_isp_remove(struct platform_device *pdev) {
    struct cam_isp_dev *isp = platform_get_drvdata(pdev);

    dma_free_coherent(&pdev->dev, isp->frame_size,
                     isp->frame_buf, isp->frame_dma);
    of_reserved_mem_device_release(&pdev->dev);
    return 0;
}
```

Summary

Concept	Key Point
Fragmentation	Free pages scattered, can't satisfy large contiguous requests
Compaction	Migrates movable pages to consolidate free space; slow but no memory waste
Migration types	<code>UNMOVABLE</code> (kernel), <code>MOVABLE</code> (user), <code>RECLAIMABLE</code> (cache), <code>CMA</code>
CMA	Reserves region at boot; reused for movable pages normally; evicted on demand
Carveout (<code>no-map</code>)	Hard reservation; always available; wastes memory
DT binding	<code>reusable</code> = CMA; <code>no-map</code> = carveout; <code>memory-region</code> = per-device binding
Driver API	<code>of_reserved_mem_device_init()</code> + <code>dma_alloc_coherent()</code>
ARM64 relevance	CMA is essential for camera, GPU, video codec on Qualcomm SoCs

Given your i.MX8X/M board bring-up and Qualcomm SoC experience, the **Device Tree** `reserved-memory` node and `of_reserved_mem_device_init()` pattern will be immediately applicable. The next natural topic would be **DMA coherency and cache management on ARM64** (IOMMU, SMMU, cache maintenance operations) — which ties directly into how CMA buffers are actually used by hardware. Would you like to continue there, or explore **IOMMU/SMMU on ARM64** specifically?

SECTION 10

DMA Coherency and Cache Management on ARM64

DMA coherency and cache management on ARM64 is one of the most nuanced and critical topics for SoC driver development. Given your Qualcomm SoC and i.MX8X/M background, this will directly apply to your daily work.

The Core Problem: CPU Cache vs. DMA Engine

Modern ARM64 SoCs have **multi-level caches** (L1/L2 per core, L3 shared). The CPU reads/writes go through these caches. But a DMA engine (camera ISP, GPU, codec, network controller) accesses **physical memory directly** — it bypasses the CPU cache entirely.

This creates a **coherency problem**:

Scenario: CPU writes data, DMA reads it

CPU Core

|

▼

[L1 Cache] → [L2 Cache] → [L3 Cache] → [DRAM]

↑

DMA Engine reads here

Problem:

CPU wrote to cache → data is in L1/L2 but NOT yet in DRAM

DMA reads DRAM → reads STALE/OLD data!

Scenario: DMA writes data, CPU reads it

DMA Engine writes to DRAM

CPU reads → hits L1 cache → reads OLD cached data!

(Cache has stale copy from before DMA wrote)

Two Fundamental Approaches to Coherency

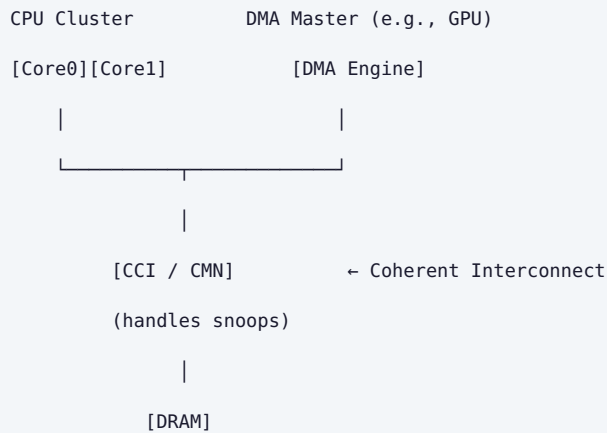
Approach 1: Hardware Cache Coherency (HW Coherent DMA)

The hardware automatically keeps caches and memory in sync. The CPU and DMA engine participate in a **cache coherency protocol** (similar to MESI on x86, or ACE/CHI on ARM).

On ARM64 SoCs with a **CCI (Cache Coherent Interconnect)** or **CMN (Coherent Mesh Network)**:

- DMA masters are connected to the interconnect as **coherent masters**
- When DMA reads, it can **snoop** CPU caches directly
- No software cache maintenance needed

ARM64 SoC with Hardware Coherency:

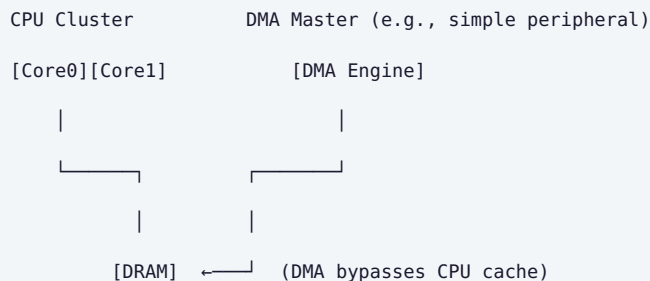


Driver implication: Use `dma_alloc_coherent()` — no explicit cache flushes needed.

Approach 2: Non-Coherent DMA (Software Cache Maintenance)

The DMA master is **not connected** to the cache coherency fabric. Software must explicitly flush/invalidate caches before/after DMA transfers.

ARM64 SoC without full coherency:

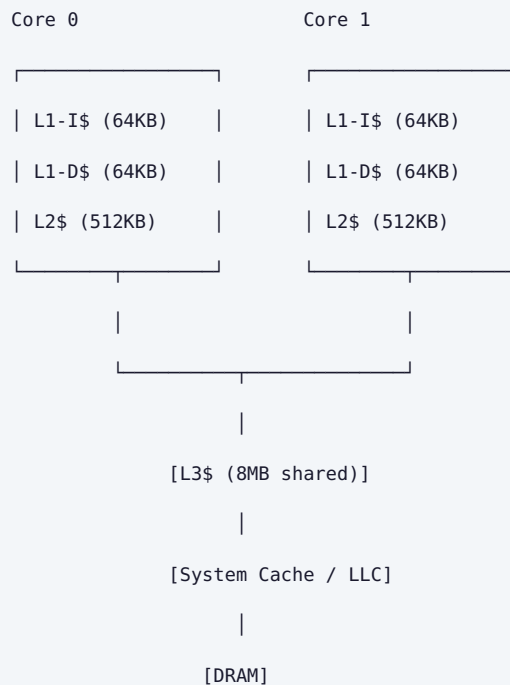


Driver implication: Use **streaming DMA APIs** with explicit direction flags.

ARM64 Cache Architecture

Before diving into APIs, understand what you're managing:

ARM64 Cache Hierarchy (typical Qualcomm SoC):



Cache Line Size

On ARM64, cache line size is typically **64 bytes** (check with `CTR_EL0`):

```
// Get cache line size in kernel
unsigned int cache_line_size(void); // returns 64 typically
```

Critical rule: DMA buffers must be **cache-line aligned** to avoid false sharing between DMA and CPU data.

ARM64 Cache Maintenance Instructions

These are the low-level instructions the kernel uses (you won't call these directly in drivers, but understanding them is essential):

Data Cache Operations

```
// Clean (write-back dirty cache lines to memory – DMA can now read correct data)
DC CVAC, Xn    // Clean by VA to Point of Coherency
DC CIVAC, Xn   // Clean and Invalidate by VA to PoC

// Invalidate (discard cache lines – CPU will re-fetch from memory after DMA write)
DC IVAC, Xn    // Invalidate by VA to PoC (use carefully – can lose data!)

// Clean to Point of Unification (for I-cache coherency)
DC CVAU, Xn

// Clean to Persistence (for persistent memory)
DC CVAP, Xn
```

DSB / DMB — Memory Barriers

```
DSB SY    // Data Synchronization Barrier – all memory accesses complete before this
DMB SY    // Data Memory Barrier – ordering only, doesn't wait for completion
ISB       // Instruction Synchronization Barrier – flush pipeline
```

In kernel C code:

```
dsb(sy);    // DSB SY
dmb(sy);    // DMB SY
isb();      // ISB
```

The Linux DMA API — Complete Picture

Category 1: Coherent DMA Mapping

Memory that is **always coherent** — CPU and DMA always see the same data, no explicit cache ops needed.

```
// Allocate coherent DMA memory
void *cpu_addr = dma_alloc_coherent(dev, size, &dma_handle, GFP_KERNEL);
```

What happens internally on ARM64:

- If hardware coherent: allocates normal cached memory, marks as coherent in IOMMU
- If not hardware coherent: allocates **uncached** memory (mapped with `MT_NORMAL_NC` in page tables) — CPU accesses bypass cache entirely

Coherent buffer (non-HW-coherent SoC):

CPU write → bypasses cache → goes directly to DRAM

DMA read → reads DRAM → always sees latest data ✓

Cost: CPU accesses are SLOW (no cache benefit)

Use for: Control structures, small descriptors, anything CPU reads/writes frequently

Category 2: Streaming DMA Mapping

For **large data buffers** where you want CPU cache benefits but need to DMA occasionally:

```
// Map an existing buffer for DMA
dma_addr_t dma_addr = dma_map_single(
    dev,
    cpu_ptr,          // kernel virtual address
    size,
    DMA_TO_DEVICE     // CPU→DMA: clean cache before DMA reads
    // DMA_FROM_DEVICE // DMA→CPU: invalidate cache after DMA writes
    // DMA_BIDIRECTIONAL // both
);

if (dma_mapping_error(dev, dma_addr))
    return -ENOMEM;

// Program dma_addr into hardware...
// Start DMA transfer...
// Wait for DMA completion...

// Unmap – makes buffer safe for CPU access again
dma_unmap_single(dev, dma_addr, size, DMA_FROM_DEVICE);

// Now safe to read CPU-side
process_data(cpu_ptr);
```

What `dma_map_single` does on ARM64 (non-coherent):

DMA_TO_DEVICE (CPU wrote data, DMA will read):

- dc civac (Clean + Invalidate cache lines)
- DSB
- CPU cache lines are written back to DRAM
- DMA engine reads correct data from DRAM ✓

DMA_FROM_DEVICE (DMA will write, CPU will read):

- dc ivac (Invalidate cache lines)
- DSB
- CPU cache lines are discarded
- After DMA writes to DRAM, CPU re-fetches fresh data ✓

DMA_BIDIRECTIONAL:

- dc civac (Clean + Invalidate)
- Used when direction is unknown

Category 3: DMA Scatter-Gather

For non-contiguous buffers (common in network drivers, storage):

```
struct scatterlist sg[NUM_ENTRIES];
sg_init_table(sg, NUM_ENTRIES);
sg_set_buf(&sg, buf0, len0);
sg_set_buf(&sg, buf1, len1);

// Map entire scatter-gather list
int nents = dma_map_sg(dev, sg, NUM_ENTRIES, DMA_TO_DEVICE);
if (!nents)
    return -ENOMEM;

// Iterate mapped entries (may be fewer than input due to merging)
struct scatterlist *s;
int i;
for_each_sg(sg, s, nents, i) {
    dma_addr_t addr = sg_dma_address(s);
    unsigned int len = sg_dma_len(s);
    // program addr+len into DMA descriptor ring
}

// After DMA completes:
dma_unmap_sg(dev, sg, NUM_ENTRIES, DMA_TO_DEVICE);
```

Category 4: DMA Sync Operations (Partial Sync)

When you need to access a DMA buffer from CPU **without unmapping** it:

```
// CPU needs to read DMA-written data (without full unmap)
dma_sync_single_for_cpu(dev, dma_addr, size, DMA_FROM_DEVICE);

// → invalidates CPU cache for this range
// → CPU can now safely read

// CPU done, DMA needs to read CPU-written data
dma_sync_single_for_device(dev, dma_addr, size, DMA_TO_DEVICE);

// → cleans CPU cache for this range
// → DMA can now safely read

// Use case: ping-pong buffers, descriptor rings
```

IOMMU and SMMU on ARM64

What Is an IOMMU?

An **IOMMU (Input-Output Memory Management Unit)** is the hardware equivalent of the CPU's MMU, but for DMA masters. It translates **device virtual addresses (IOVA)** to physical addresses.

On ARM, the IOMMU is called **SMMU (System Memory Management Unit)** — ARM SMMU v2 or SMMUv3.

Without IOMMU:

DMA Engine → physical address → DRAM

(DMA can access ANY physical address – security risk!)

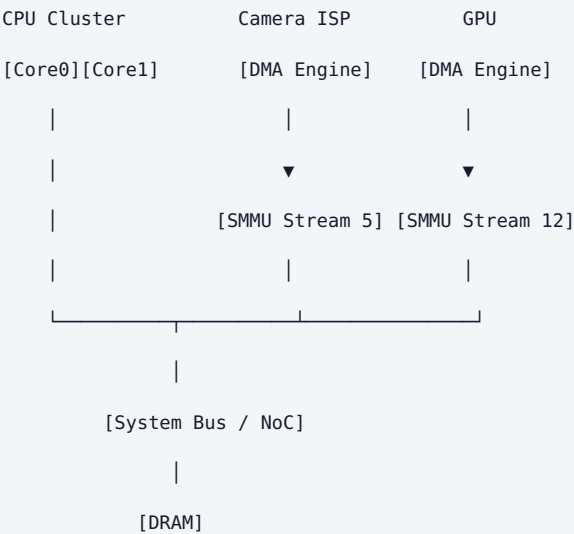
With IOMMU/SMMU:

DMA Engine → IOVA → [SMMU translates] → physical address → DRAM

(DMA can only access addresses mapped in its SMMU page table)

SMMU Architecture

ARM64 SoC with SMMU:



Each device has its own SMMU "stream" with its own page table.
Camera ISP can only access memory mapped in Stream 5's page table.
GPU can only access memory mapped in Stream 12's page table.

SMMU Benefits

- Security/Isolation** — A compromised DMA master can't access arbitrary physical memory
- IOVA space** — Devices see a flat 64-bit address space regardless of physical layout
- Scatter-gather in hardware** — Non-contiguous physical pages appear contiguous to device
- Fault isolation** — SMMU faults are reported per-device, not system-wide crashes

SMMU in Device Tree (Qualcomm SoC)

```

smmu: iommu@15000000 {
    compatible = "arm,smmu-v2";
    reg = <0x15000000 0x10000>;

    #iommu-cells = <1>;

    interrupts = <GIC_SPI 48 IRQ_TYPE_LEVEL_HIGH>;
};

camera_isp: isp@1a000000 {
    compatible = "qcom,camera-isp";
    reg = <0x1a000000 0x10000>;

    iommus = <&smmu 0x200>; /* bind to SMMU stream 0x200 */
};

```

SMMU Driver Interaction

When your driver calls `dma_alloc_coherent()` or `dma_map_single()`, the DMA API automatically programs the SMMU:

```

// This call:
dma_addr_t iova = dma_map_single(dev, cpu_ptr, size, DMA_TO_DEVICE);

// Internally does:
// 1. Allocate an IOVA from the device's IOVA space
// 2. Create SMMU page table entries: IOVA → physical pages
// 3. Perform cache maintenance (clean/invalidate)
// 4. Return IOVA to driver (this is what you program into hardware)

// The device sees 'iova', SMMU translates to physical address

```

SMMU Fault Handling

```
// SMMU faults appear as kernel messages:

// arm-smmu 15000000.iommu: Unhandled context fault
// arm-smmu 15000000.iommu: IOVA=0x12345678, FSYNR=0x00000002
// arm-smmu 15000000.iommu: SID=0x200 (camera_isp)

// Common causes:
// 1. Programming wrong IOVA into hardware register
// 2. Accessing unmapped IOVA (forgot dma_map_single)
// 3. Buffer overflow past mapped region
// 4. Using physical address instead of IOVA
```

Cache Attributes and Memory Types on ARM64

ARM64 page table entries encode **memory type** via MAIR_EL1 register:

Memory Type	Cache Behavior	Use Case
MT_NORMAL	Fully cached (WB/WA)	Normal RAM, kernel data
MT_NORMAL_NC	Non-cacheable	Coherent DMA buffers (non-HW-coherent)
MT_NORMAL_WT	Write-through	Rarely used
MT_DEVICE_nGnRnE	Strongly ordered, no reorder	MMIO registers
MT_DEVICE_nGnRE	Device memory	PCIe MMIO
MT_DEVICE_GRE	Gathering, reordering allowed	Write-combining (framebuffers)

In Driver Code — `ioremap` Variants

```
// Map device registers (strongly ordered, no cache)
void __iomem *base = ioremap(phys_addr, size);
// Uses MT_DEVICE_nGnRE

// Map write-combining memory (framebuffer, GPU VRAM)
void __iomem *fb = ioremap_wc(phys_addr, size);
// Uses MT_NORMAL_NC or write-combining attributes

// Map cached (unusual for MMIO, but used for some SoC peripherals)
void __iomem *cached = ioremap_cache(phys_addr, size);

// Unmap
iounmap(base);
```

Complete DMA Coherency Decision Tree

```

Need to DMA to/from a buffer?
|
├─ Is the SoC hardware coherent? (check DT: dma-coherent property)
|   └─ YES → dma_alloc_coherent() or dma_map_single() with no manual cache ops
|       SMMU handles address translation, HW handles cache coherency
|   └─ NO → continue below
|
├─ Is the buffer always shared between CPU and DMA simultaneously?
|   └─ YES → dma_alloc_coherent()
|       (non-cached mapping, always coherent, slower CPU access)
|
├─ Is the buffer large (video frame, audio buffer)?
|   └─ YES → Streaming DMA:
|       dma_map_single() before DMA
|       dma_unmap_single() after DMA
|       CPU accesses only when unmapped
|
├─ Is the buffer non-contiguous (scatter-gather)?
|   └─ YES → dma_map_sg() / dma_unmap_sg()
|
└─ Need to access buffer from CPU without unmapping?
    └─ YES → dma_sync_single_for_cpu() / dma_sync_single_for_device()
  
```

Practical Rules for ARM64 Driver Development

Rule 1: Never access a DMA buffer from CPU while DMA is active

```
// WRONG

dma_map_single(dev, buf, size, DMA_FROM_DEVICE);

start_dma();

printk("%x", buf); // BUG: DMA may be writing, cache is invalid!


// RIGHT

dma_map_single(dev, buf, size, DMA_FROM_DEVICE);

start_dma();

wait_for_dma_completion();

dma_unmap_single(dev, dma_addr, size, DMA_FROM_DEVICE);

printk("%x", buf); // Safe: cache invalidated by unmap
```

Rule 2: Always check `dma_mapping_error()`

```
dma_addr_t dma_addr = dma_map_single(dev, buf, size, DMA_TO_DEVICE);

if (dma_mapping_error(dev, dma_addr)) {

    dev_err(dev, "DMA mapping failed

");

    return -ENOMEM;

}
```

Rule 3: Match map/unmap directions

```
// Map for device read

dma_addr_t addr = dma_map_single(dev, buf, size, DMA_TO_DEVICE);

// ... DMA reads buf ...

dma_unmap_single(dev, addr, size, DMA_TO_DEVICE); // Same direction!
```

Rule 4: DMA buffers must be cache-line aligned

```
// WRONG – may share cache line with other data
char buf[100];
dma_map_single(dev, buf, 100, DMA_FROM_DEVICE);

// RIGHT – cache-line aligned allocation
char *buf = kmalloc(ALIGN(100, dma_get_cache_alignment()), GFP_KERNEL);
// Or use dma_alloc_coherent which handles alignment automatically
```

Rule 5: Check `dma-coherent` in Device Tree

```
// If your SoC has hardware coherency:
soc {
    dma-coherent; // All DMA masters are hardware coherent
};

// Or per-device:
camera_isp: isp@1a00000 {
    dma-coherent;
};
```

In driver:

```
// Check at runtime
if (dev_is_dma_coherent(dev)) {
    // No cache maintenance needed
} else {
    // Must use streaming DMA APIs with explicit sync
}
```

SMMU v3 — Modern ARM64 (Qualcomm Snapdragon 8 Gen series)

SMMUv3 (used in modern Qualcomm SoCs) adds:

- **2-stage translation:** Stage 1 (device IOVA → IPA) + Stage 2 (IPA → PA) for virtualization
- **PCIe ATS support:** Device-side TLB for performance
- **PRI (Page Request Interface):** Device can request page faults be handled (like CPU page faults)

- **SVA (Shared Virtual Addressing):** Device shares CPU's page table — device uses same virtual addresses as CPU process

```
// SVA – device uses same virtual addresses as userspace process
// (Used in Qualcomm DSP/NPU drivers for ML inference)
struct iommu_sva *handle = iommu_sva_bind_device(dev, current->mm, NULL);
u32 pasid = iommu_sva_get_pasid(handle);
// Program PASID into device – device now shares process address space
// Device can access any userspace virtual address directly!
iommu_sva_unbind_device(handle);
```

Debugging DMA and Cache Issues

```
# Enable DMA API debug (catches mapping errors, leaks)
CONFIG_DMA_API_DEBUG=y
CONFIG_DMA_API_DEBUG_SG=y

# SMMU fault messages in dmesg
dmesg | grep -i smmu
dmesg | grep -i iommu

# Check IOMMU groups
ls /sys/kernel/iommu_groups/

# DMA mapping statistics
cat /sys/kernel/debug/dma-api/dump

# ARM64 cache info
cat /sys/devices/system/cpu/cpu0/cache/index*/size
cat /sys/devices/system/cpu/cpu0/cache/index*/coherency_line_size

# Check if device is behind SMMU
cat /sys/bus/platform/devices/<device>/iommu_group/type
```

Summary Reference

Concept	Key Point
Cache coherency problem	CPU cache and DMA engine see different data
HW coherent	Interconnect (CCI/CMN) snoops caches; no SW cache ops needed
Non-coherent	SW must clean/invalidate caches via DMA API
<code>dma_alloc_coherent</code>	Always coherent; non-cached on non-HW-coherent SoCs
<code>dma_map_single</code>	Streaming; clean cache before DMA read, invalidate before CPU read
SMMU	ARM's IOMMU; translates device IOVA → physical; provides isolation
<code>MT_DEVICE_nGnRnE</code>	Memory type for MMIO registers (strongly ordered)
<code>MT_NORMAL_NC</code>	Non-cached normal memory for coherent DMA buffers
<code>dma-coherent</code> DT property	Marks SoC/device as hardware coherent
SMMUv3 SVA	Device shares CPU process page table (used in NPU/DSP drivers)
Key files	<code>drivers/iommu/arm/arm-smmu-v3/</code> , <code>kernel/dma/</code> , <code>arch/arm64/mm/dma-mapping.c</code>

This completes the full memory management chain: **Buddy Allocator → SLUB → CMA → DMA Coherency → SMMU**. The next natural progression would be **writing a complete platform device driver** that ties all of this together — using CMA for DMA buffers, SMMU for address translation, and proper cache management. Or we could move into **interrupt handling on ARM64** (GIC, IRQ domains, threaded IRQs) which is the other major pillar of driver development. Which direction would you like to go?

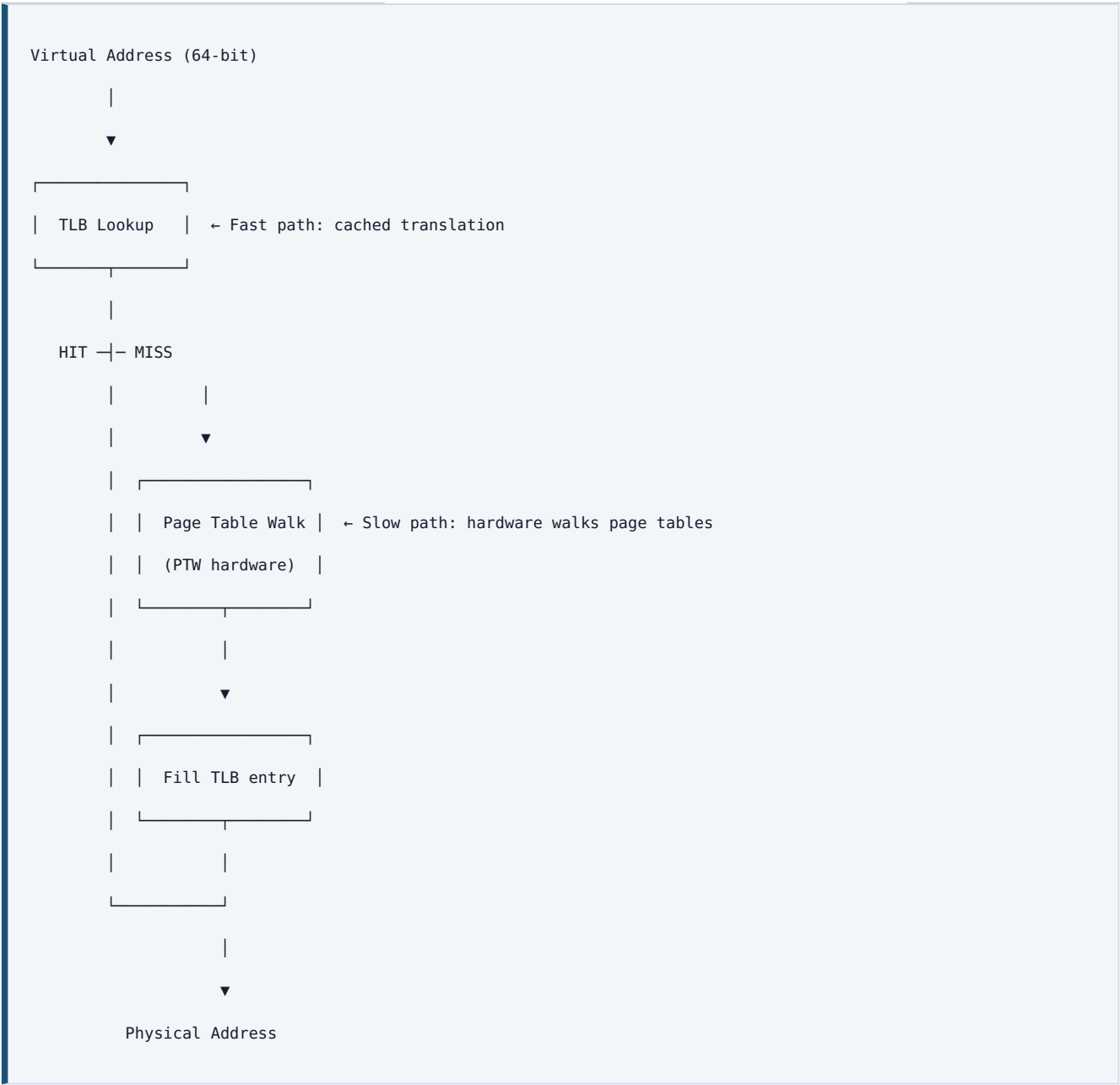
SECTION 11

MMU: Virtual-to-Physical Address Translation — ARM64

This is one of the most fundamental and satisfying topics in ARM64 architecture — the full virtual-to-physical address translation pipeline. Let me walk through every stage in detail.

| Overview: The Translation Pipeline

When the CPU executes an instruction that accesses a virtual address, the hardware performs a **page table walk** to find the corresponding physical address. This is done by the **MMU (Memory Management Unit)**, accelerated by the **TLB (Translation Lookaside Buffer)**.



ARM64 Virtual Address Space Layout

ARM64 uses a **split virtual address space** controlled by two translation table base registers:

Virtual Address Space (48-bit, VA_BITS=48):

```

0xFFFF_FFFF_FFFF_FFFF 7
                        | Kernel Space (TTBR1_EL1)
0xFFFF_0000_0000_0000 J (top 128 TB)

[HOLE – invalid addresses]
(bits 63:48 must all be 0 or all 1)

0x0000_FFFF_FFFF_FFFF 7
                        | User Space (TTBR0_EL1)
0x0000_0000_0000_0000 J (bottom 128 TB)

```

How the CPU decides which TTBR to use:

- If VA bits [63:48] are all **0** → use **TTBR0_EL1** (user space)
- If VA bits [63:48] are all **1** → use **TTBR1_EL1** (kernel space)
- Any other pattern → **Translation fault** (the "canonical address" rule)

```

// Linux kernel defines:

#define PAGE_OFFSET 0xFFFF800000000000UL // Start of kernel linear map

#define TASK_SIZE   0x0000FFFFFFFFF0000UL // Max user space address

```

TCR_EL1 — Translation Control Register

This register configures the entire translation system. Critical fields:

TCR_EL1:



T0SZ: Size offset for TTBR0 region (user space)

VA size = $2^{(64 - T0SZ)}$

T0SZ=16 → 48-bit user VA space

T1SZ: Size offset for TTBR1 region (kernel space)

T1SZ=16 → 48-bit kernel VA space

TG0/TG1: Translation Granule (page size)

00 = 4KB ← most common on Linux ARM64

01 = 64KB

10 = 16KB ← used on Apple Silicon

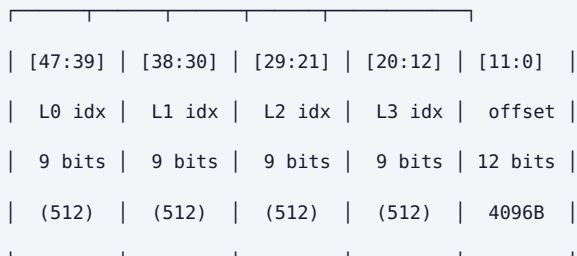
SH0/SH1: Shareability (Inner Shareable for normal memory)

IRGN/ORGN: Cache attributes for page table walks

ARM64 Page Table Structure — 4KB Granule, 48-bit VA

With **4KB pages** and **48-bit virtual addresses**, ARM64 uses a **4-level page table**:

Virtual Address (48 bits):



Each level indexes into a **512-entry table** (9 bits = $2^9 = 512$ entries, each entry 8 bytes = 4KB per table).

Level	ARM64 Name	Linux Name	Covers	Entry Maps
0	L0	PGD	512 GB per entry	Points to L1 table
1	L1	PUD	1 GB per entry	Points to L2 table (or 1GB block)
2	L2	PMD	2 MB per entry	Points to L3 table (or 2MB block)
3	L3	PTE	4 KB per entry	Points to physical page

Let's walk through translating virtual address `0xFFFF800012345678` (a kernel address).

```
VA = 0xFFFF800012345678
```

Bits [63:48] = 0xFFFF → all 1s → use TTBR1_EL1 (kernel)

```
TTBR1_EL1 = 0x0000 0000 8000 0000 (physical address of PGD table)
```

```

VA = 0xFFFF 8000 1234 5678

Binary: 1111111111111111 10000000000000000010010001101000101011001111000

Bits [47:39] = L0 index = 0b100000000 = 256
Bits [38:30] = L1 index = 0b000000000 = 0
Bits [29:21] = L2 index = 0b000010010 = 18
Bits [20:12] = L3 index = 0b001101000 = 104
Bits [11:0]  = offset   = 0b010101100111000 = 0x678

```

Step 2: L0 (PGD) Lookup

PGD base = TTBR1_EL1 = 0x80000000 (physical)

L0 index = 256

Entry address = 0x80000000 + (256 × 8) = 0x80000800

Read 8 bytes at physical 0x80000800:

PGD entry = 0x0000_0000_8010_0003

Decode PGD entry:

Bits [1:0] = 0b11 → Valid Table Descriptor

Bits [47:12] = 0x80100 → Next level table at physical 0x80100000

Step 3: L1 (PUD) Lookup

PUD base = 0x80100000 (from PGD entry)

L1 index = 0

Entry address = 0x80100000 + (0 × 8) = 0x80100000

Read 8 bytes at physical 0x80100000:

PUD entry = 0x0000_0000_8020_0003

Decode:

Bits [1:0] = 0b11 → Valid Table Descriptor

Next level table at physical 0x80200000

Step 4: L2 (PMD) Lookup

```
PMD base = 0x80200000
```

```
L2 index = 18
```

```
Entry address = 0x80200000 + (18 × 8) = 0x80200090
```

```
Read 8 bytes at physical 0x80200090:
```

```
PMD entry = 0x0000_0000_8030_0003
```

```
Decode:
```

```
Bits [1:0] = 0b11 → Valid Table Descriptor (not a 2MB block)
```

```
Next level table at physical 0x80300000
```

```
(If bits [1:0] = 0b01 → Block descriptor → 2MB huge page, walk stops here)
```

Step 5: L3 (PTE) Lookup

PTE base = 0x80300000

L3 index = 104

Entry address = 0x80300000 + (104 × 8) = 0x80300340

Read 8 bytes at physical 0x80300340:

PTE entry = 0x0060_0000_9000_0713

Decode PTE entry (this is the most important step):

Bits [1:0] = 0b11 → Valid Page Descriptor

Bit = AF (Access Flag) = 1 → page has been accessed

Bits [9:8] = SH (Shareability) = 0b11 → Inner Shareable

Bits [7:6] = AP (Access Permissions) = 0b01 → EL0 read-only, EL1 RW

Bit = NS (Non-Secure) = 0

Bits [4:2] = AttrIdx = 0b100 → index into MAIR_EL1 (memory type)

Bits [47:12] = Output Address = 0x90000 → physical page at 0x90000000

Bit = DBM (Dirty Bit Modifier)

Bit = Contiguous hint

Bit = PXN (Privileged Execute Never)

Bit = UXN/XN (Unprivileged Execute Never)

Step 6: Form Physical Address

Physical page base = PTE output address = 0x90000000

Page offset = VA bits [11:0] = 0x678

Physical Address = 0x90000000 + 0x678 = 0x90000678 ✓

Full Walk Diagram

VA: 0xFFFF800012345678

|

| bits[63:48]=0xFFFF → TTBR1_EL1

|

▼

TTBR1_EL1 → [PGD Table @ 0x80000000]

|

| index = bits[47:39] = 256

| entry @ 0x80000800

▼

[PUD Table @ 0x80100000]

|

| index = bits[38:30] = 0

| entry @ 0x80100000

▼

[PMD Table @ 0x80200000]

|

| index = bits[29:21] = 18

| entry @ 0x80200090

▼

[PTE Table @ 0x80300000]

|

| index = bits[20:12] = 104

| entry @ 0x80300340

▼

Physical Page @ 0x90000000

|

| + offset bits[11:0] = 0x678

▼

Physical Address: 0x90000678 ✓

AF=0: Page has never been accessed → triggers Access Flag Fault on first access

Hardware sets AF=1 automatically (if FEAT_HAFDBS is supported)

Or kernel's fault handler sets it in software

AF=1: Page has been accessed (used by page reclaim – LRU tracking)

Dirty Bit Modifier (DBM, bit 51):

DBM=1: Hardware automatically sets AP to indicate page is dirty (written)

Used for hardware dirty page tracking (FEAT_HAFDBS)

Kernel uses this for efficient page reclaim and CoW

AttrIdx [4:2] — Memory Type:

AttrIdx indexes into MAIR_EL1 register (8 slots):

MAIR_EL1 typical Linux ARM64 setup:

Index 0: MT_DEVICE_nGnRE (strongly ordered MMIO)

Index 1: MT_DEVICE_nGnRE (device memory)

Index 2: MT_DEVICE_GRE (write-combining)

Index 3: MT_NORMAL_NC (non-cacheable normal)

Index 4: MT_NORMAL (normal cached WB/WA) ← most RAM uses this

Index 5: MT_NORMAL_WT (write-through)

TLB — Translation Lookaside Buffer

The page table walk above requires **4 memory accesses** (one per level). This would be catastrophically slow if done for every memory access. The **TLB** caches recent translations.

TLB Structure on ARM64

ARM64 TLB (conceptual):

ASID	Virtual Page Number	Physical Page Number
(16b)	(VA bits [63:12])	(PA bits [47:12])
		+ attributes
0x42	0xFFFF800012345	0x90000 + RW, cached
0x42	0x0000000000401	0xA1000 + R0, cached
0x17	0x0000000000401	0xB2000 + RW, cached
...	...	...

ASID — Address Space Identifier

Without ASIDs, every context switch (process switch) would require a **full TLB flush** — catastrophically expensive. ASIDs solve this:

- Each process gets a unique **ASID** (16-bit on ARM64 with `FEAT_ASID16`)
- TLB entries are tagged with ASID
- On context switch: just update TTBR0_EL1 with new process's page table + new ASID
- Old TLB entries with different ASID are **ignored** (not flushed!)
- Only when ASIDs are exhausted (after 65536 processes) does the kernel do a full TLB flush and reassign ASIDs

```
// Linux ARM64 context switch (simplified):
void cpu_switch_mm(pgd_t *pgd, struct mm_struct *mm) {
    unsigned long ttbr1 = read_sysreg(ttbr1_el1);

    unsigned long asid = ASID(mm); // get or assign ASID

    // Write new TTBR0 with ASID atomically
    write_sysreg(ttbr0 | (asid << 48), ttbr0_el1);

    isb();
}
```

TLB Lookup Flow

```

CPU accesses VA 0xFFFF800012345678
|
▼
TLB lookup: search for (ASID=kernel, VPN=0xFFFF800012345)
|
├─ HIT: return PA=0x90000678 + attributes
|   → memory access proceeds immediately (~1 cycle)
|
└─ MISS: hardware Page Table Walker activates
    |
    ├─ Reads TTBR1_EL1 (kernel, no ASID needed)
    ├─ Walks L0 → L1 → L2 → L3 (4 memory reads)
    ├─ Checks permissions → fault if violation
    ├─ Fills TLB entry
    └─ Retries memory access (~10-100 cycles)

```

TLB Miss Penalty

```

L1 cache hit:    ~4 cycles
TLB hit:         ~1 cycle (parallel with L1 cache lookup)
TLB miss:        ~10-100 cycles (page table walk, all in L1/L2 cache ideally)
TLB miss + page fault: ~1000+ cycles (kernel fault handler)
TLB miss + major fault: ~millions of cycles (disk I/O)

```

TLB Maintenance Instructions

The kernel must explicitly manage the TLB in certain situations:

When to Flush TLB

1. Page table entry changed (permissions, PA changed)
2. Page unmapped (PTE cleared)
3. Process exit (all user mappings gone)
4. ASID rollover
5. ioremap / iounmap
6. mprotect() system call

ARM64 TLB Invalidation Instructions

```
// Invalidate by VA, all ASIDs, Inner Shareable
TLBI VAAE1IS, Xn    // Xn = VA >> 12 (page-aligned)

// Invalidate by VA, specific ASID
TLBI VAE1IS, Xn     // Xn = (ASID << 48) | (VA >> 12)

// Invalidate all TLB entries for EL1 (kernel), Inner Shareable
TLBI VMALLE1IS

// Invalidate all TLB entries for EL0 (user), specific ASID
TLBI ASIDE1IS, Xn   // Xn = ASID << 48

// DSB required after TLBI to ensure completion
DSB ISH
ISB
```

In Linux Kernel Code

```
// Flush single VA (user space)
flush_tlb_page(vma, addr);

// Flush range of VAs
flush_tlb_range(vma, start, end);

// Flush entire user address space (process exit)
flush_tlb_mm(mm);

// Flush kernel mapping
flush_tlb_kernel_range(start, end);

// ARM64 implementation in arch/arm64/include/asm/tlbflush.h:
static inline void __flush_tlb_page(struct mm_struct *mm, unsigned long uaddr) {
    unsigned long addr = __TLBI_VADDR(uaddr, ASID(mm));
    dsb(ishst);
    __tlbi(vale1is, addr); // TLBI VAE1IS
    dsb(ish);
}
```

Huge Pages — Block Descriptors (Shortcut the Walk)

ARM64 supports **block descriptors** at L1 and L2 levels, which map large regions without needing lower-level tables:

L1 Block Descriptor (1 GB page):

bits[1:0] = 0b01 → Block

Walk stops at L1 → maps 1 GB directly

VA bits [29:0] = offset within 1 GB block

L2 Block Descriptor (2 MB page):

bits[1:0] = 0b01 → Block

Walk stops at L2 → maps 2 MB directly

VA bits [20:0] = offset within 2 MB block

Normal 4KB walk: L0 → L1 → L2 → L3 → PA (4 levels)

2MB huge page walk: L0 → L1 → L2[block] → PA (3 levels, stops at L2)

1GB huge page walk: L0 → L1[block] → PA (2 levels, stops at L1)

Linux uses 2MB huge pages for:

- Kernel linear map (most of physical RAM mapped as 2MB blocks)
- Transparent Huge Pages (THP) for user processes
- `mmap(MAP_HUGETLB)` for explicit huge page allocation

Benefit: Fewer TLB entries needed. One 2MB TLB entry covers what would otherwise require 512 separate 4KB TLB entries.

Complete ARM64 Translation — All Configurations

Page Size	VA Bits	Levels	L0 covers	L1 covers	L2 covers	L3 covers
4KB	48-bit	4	256 TB	512 GB	1 GB	2 MB → 4KB
4KB	39-bit	3	(skipped)	512 GB	1 GB	2 MB → 4KB
16KB	48-bit	4	128 PB	64 TB	32 GB	64 MB → 16KB
64KB	52-bit	3	(skipped)	4 PB	512 MB	64 MB → 64KB

Linux on ARM64 typically uses **4KB pages, 48-bit VA, 4 levels** (the first row).

Putting It All Together — What Happens on Every Memory Access

CPU executes: `LDR X0, [X1]` (load from address in X1)

1. MMU checks: is VA in X1 canonical? (bits[63:48] all 0 or all 1)

NO → Translation fault (SIGSEGV)

2. MMU selects TTBR: bits = 1 → `TTBR1_EL1` (kernel)

3. TLB lookup: search (ASID, VPN)

HIT → get PA + attributes → go to step 7

MISS → go to step 4

4. Hardware Page Table Walker reads `TTBR1_EL1` → PGD base address

5. Walk `L0` → `L1` → `L2` → `L3`:

Each level: read 8-byte descriptor from physical memory

Check valid bit → invalid → Translation fault

Check block bit → block → stop early (huge page)

6. At `L3`: read PTE

Check valid bit → invalid → Translation fault (demand paging)

Check AF bit → `AF=0` → Access Flag fault (kernel sets `AF=1`, retries)

Check AP bits → permission violation → Permission fault (SIGSEGV or kernel oops)

Extract physical page address

7. Form PA: `physical_page_base | VA[11:0]`

8. Check memory type (`AttrIdx` → `MAIR_EL1`):

`MT_NORMAL` → access through cache

`MT_DEVICE` → bypass cache, strongly ordered

9. Fill TLB entry with (ASID, VPN, PPN, attributes)

10. Complete memory access: load/store to physical address

Fault Types and Linux Handlers

When the page table walk fails, the CPU raises a **synchronous exception**:

ESR_EL1 (Exception Syndrome Register) encodes the fault:

EC field [31:26]:

0b100100 → Data Abort from EL0 (user space memory access fault)

0b100101 → Data Abort from EL1 (kernel memory access fault)

0b100000 → Instruction Abort from EL0

0b100001 → Instruction Abort from EL1

DFSC field [5:0] (Data Fault Status Code):

0b000000 → Address size fault, L0

0b000001 → Address size fault, L1

0b000100 → Translation fault, L0 ← page not mapped

0b000101 → Translation fault, L1

0b000110 → Translation fault, L2

0b000111 → Translation fault, L3 ← most common (PTE not present)

0b001001 → Access flag fault, L1

0b001011 → Access flag fault, L3 ← AF=0, first access

0b001101 → Permission fault, L1

0b001111 → Permission fault, L3 ← write to read-only page (CoW!)

FAR_EL1: The virtual address that caused the fault

ELR_EL1: The instruction address that caused the fault (return address)

Linux ARM64 fault dispatch:

```
// arch/arm64/mm/fault.c

void do_mem_abort(unsigned long far, unsigned long esr,
                  struct pt_regs *regs) {
    const struct fault_info *inf = esr_to_fault_info(esr);
    inf->fn(far, esr, regs); // dispatch to specific handler
}

// Handlers:
do_translation_fault() → demand paging, CoW, stack growth
do_page_fault()       → calls handle_mm_fault() → VMA lookup → allocate page
do_bad_area()         → SIGSEGV (invalid address)
do_alignment_fault()  → unaligned access (rare on ARM64)
```

Key Kernel Data Structures

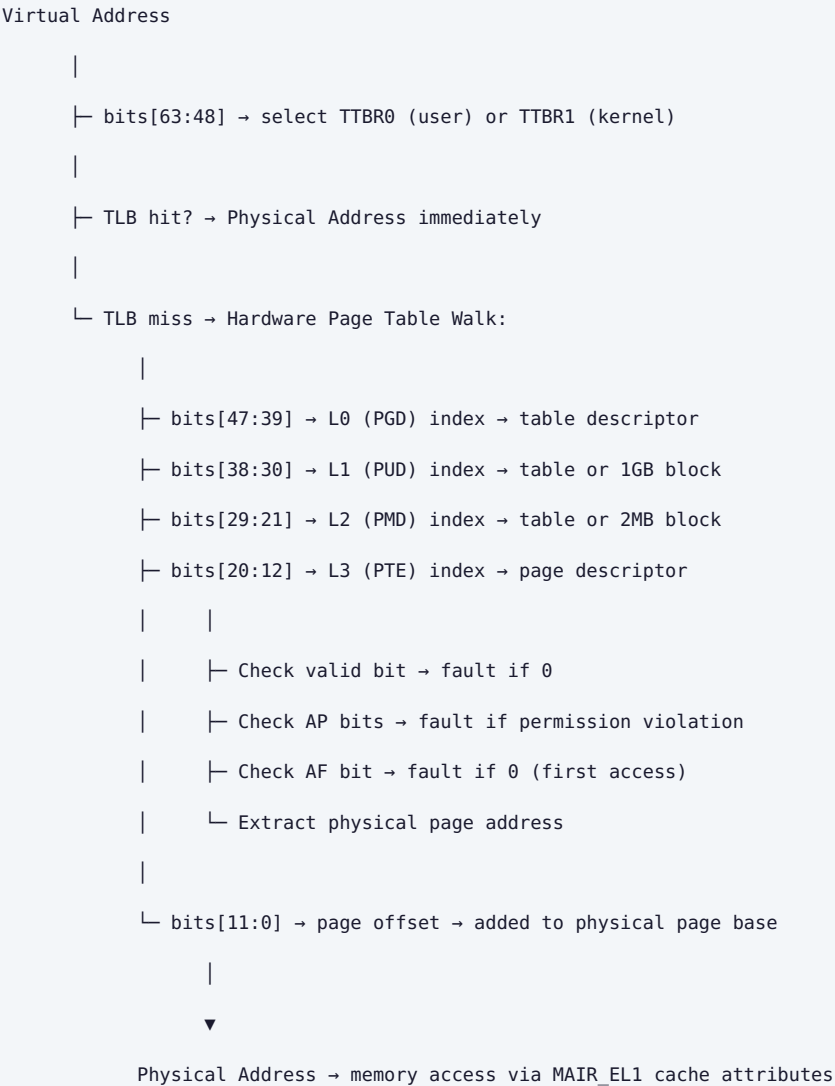
```
// Process memory descriptor
struct mm_struct {
    pgd_t *pgd;           // Physical address of L0 page table
    unsigned long mmap_base; // Start of mmap region
    struct vm_area_struct *mmap; // List of VMAs
    // ...
};

// Virtual Memory Area – describes a contiguous virtual region
struct vm_area_struct {
    unsigned long vm_start; // Start VA
    unsigned long vm_end;   // End VA
    pgprot_t vm_page_prot;  // Page protection flags
    unsigned long vm_flags; // VM_READ, VM_WRITE, VM_EXEC, VM_SHARED...
    struct file *vm_file;   // Backing file (NULL for anonymous)
    // ...
};

// Page Table Entry types (ARM64)
typedef u64 pgd_t; // L0 entry
typedef u64 pud_t; // L1 entry
typedef u64 pmd_t; // L2 entry
typedef u64 pte_t; // L3 entry

// Macros to manipulate PTEs
pte_val(pte)        // get raw u64 value
pte_present(pte)    // is page mapped?
pte_write(pte)      // is page writable?
pte_dirty(pte)      // has page been written?
pte_young(pte)      // has page been accessed (AF bit)?
pte_mkdirty(pte)    // set writable
pte_mkdirty(pte)    // set dirty
pte_mkyoung(pte)    // set AF bit
```

Summary — The Complete Picture



Register	Purpose
TTBR0_EL1	User space page table base + ASID
TTBR1_EL1	Kernel space page table base
TCR_EL1	VA size, page size, cache attributes for walks
MAIR_EL1	Memory attribute index array (8 memory types)
ESR_EL1	Exception syndrome (fault type + level)
FAR_EL1	Fault address register (VA that faulted)
ELR_EL1	Exception link register (faulting instruction VA)

For your ARM64 kernel work, the key source files are:

- `arch/arm64/include/asm/pgtable.h` — PTE manipulation macros
- `arch/arm64/mm/fault.c` — fault handler dispatch
- `arch/arm64/mm/mmu.c` — kernel page table setup
- `arch/arm64/include/asm/tlbflush.h` — TLB maintenance
- `arch/arm64/include/asm/pgtable-hwdef.h` — hardware PTE bit definitions

Would you like to continue with **how Linux sets up the initial kernel page tables during boot** (the `__create_page_tables` path in `head.S`), or move to **interrupt handling on ARM64** (GIC, exception vectors, IRQ domains)?

SECTION 12

MMU: Virtual-to-Physical Address Translation — ARM32 (ARMv7)

This is a deep and important topic — ARM 32-bit (ARMv7) has a fundamentally different memory architecture from ARM64, and understanding both side-by-side will solidify your mental model. I'll cover:

ARM32 virtual address space and page table format

How Linux sets up initial kernel page tables during boot on ARM32

Full VA→PA translation on ARM32 with TLB

Part 1: ARM32 Virtual Address Space

ARM32 uses a **32-bit virtual address space** (4 GB total), split between user and kernel:

```
ARM32 Virtual Address Space (classic 3G/1G split):

0xFFFFFFFF ↴
      | Kernel Space (1 GB)
0xC0000000 ↵ ← PAGE_OFFSET (PHYS_OFFSET mapped here)

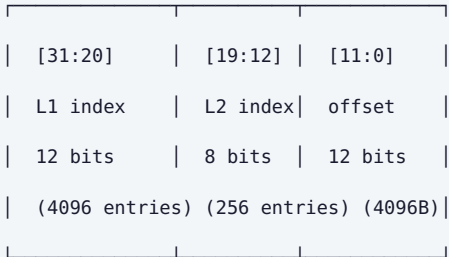
0xBFFFFFFF ↴
      | User Space (3 GB)
0x00000000 ↵
```

Unlike ARM64's two TTBRs, ARM32 uses a **single page table** per process that covers the entire 4 GB — the top 1 GB (kernel) entries are **shared across all processes** (via the global PGD).

Part 2: ARM32 Page Table Format

ARM32 uses a **2-level page table** (much simpler than ARM64's 4-level):

Virtual Address (32 bits):



Level	ARM32 Name	Linux Name	Entries	Entry Size	Table Size
1	First-level (PGD)	PGD	4096	4 bytes	16 KB
2	Second-level (PTE)	PTE	256	4 bytes	1 KB

Key difference from ARM64:

- ARM32 PGD is **16 KB** (4096×4 bytes) — must be 16 KB aligned
- ARM64 PGD is **4 KB** (512×8 bytes)
- ARM32 has only **2 levels** vs ARM64's **4 levels**

ARM32 Section Mapping (Huge Pages — 1 MB)

ARM32 supports a shortcut: if the L1 entry is a **Section descriptor**, it maps **1 MB directly** without needing an L2 table:

L1 Section Descriptor:

bits[1:0] = 0b10 → Section
bits[31:20] = Physical base address (1 MB aligned)
Walk stops at L1 → maps 1 MB directly

L1 Page Table Descriptor:

bits[1:0] = 0b01 → Coarse page table
bits[31:10] = L2 table base address
Walk continues to L2

Linux uses 1 MB section mappings for the kernel linear map — this is why the kernel boot mapping is fast to set up.

Part 3: ARM32 Page Table Entry Formats

L1 Section Descriptor (1 MB mapping)

Bits [31:20]: Section base address (physical, 1 MB aligned)

Bit : NS (Non-Secure)

Bit : 0

Bit : nG (not Global – ASID tagged if 1)

Bit : S (Shareable)

Bit : APX (Access Permission Extension)

Bits [14:12]: TEX (Type Extension – memory type)

Bits [11:10]: AP (Access Permissions)

Bits [8:5]: Domain (0–15, access control domain)

Bit : XN (Execute Never)

Bit : C (Cacheable)

Bit : B (Bufferable)

Bits [1:0]: 0b10 (Section descriptor)

L2 Small Page Descriptor (4 KB mapping)

Bits [31:12]: Physical page base address (4 KB aligned)

Bit : nG (not Global)

Bit : S (Shareable)

Bit : APX

Bits [8:6]: TEX

Bits [5:4]: AP

Bit : C

Bit : B

Bit : 1 (Small page)

Bit : XN (Execute Never)

Access Permissions (AP + APX) on ARM32

APX	AP[1:0]	Privileged (kernel)	Unprivileged (user)
0	00	No access	No access
0	01	Read/Write	No access
0	10	Read/Write	Read-only
0	11	Read/Write	Read/Write
1	00	Reserved	
1	01	Read-only	No access
1	10	Read-only	Read-only
1	11	Read-only	Read-only

Memory Type (TEX + C + B) on ARM32

TEX	C	B	Memory Type
000	0	0	Strongly Ordered (MMIO registers)
000	0	1	Shared Device
000	1	0	Write-Through, no Write-Allocate
000	1	1	Write-Back, no Write-Allocate
001	0	0	Non-cacheable
001	1	1	Write-Back, Write-Allocate ← normal RAM
010	0	0	Non-Shared Device

Part 4: ARM32 System Registers

Register	Purpose
TTBR0	User space page table base (process-specific)
TTBR1	Kernel space page table base (shared)
TTBCR	Translation Table Base Control Register
DACR	Domain Access Control Register
SCTLR	System Control Register (MMU enable bit)
DFSR	Data Fault Status Register (fault type)
DFAR	Data Fault Address Register (faulting VA)
IFSR	Instruction Fault Status Register
IFAR	Instruction Fault Address Register
CONTEXTIDR	Context ID Register (ASID, 8-bit)

TTBCR — The Split Register

ARM32 can optionally split the VA space between TTBR0 and TTBR1 (similar to ARM64):

TTBCR.N field (bits [2:0]):

N=0: TTBR0 covers entire 4 GB (classic mode, one page table)

N=1: TTBR0 covers 0x00000000–0x7FFFFFFF (2 GB user)

TTBR1 covers 0x80000000–0xFFFFFFFF (2 GB kernel)

N=2: TTBR0 covers 0x00000000–0x3FFFFFFF (1 GB user)

TTBR1 covers 0x40000000–0xFFFFFFFF (3 GB kernel)

Linux ARM32 uses N=0 (single TTBR0) with shared kernel PGD entries

DACR — Domain Access Control

ARM32 has a unique **domain** concept (16 domains, 2 bits each in DACR):

DACR bits per domain:

00 = No access (fault on any access)
01 = Client (check page table permissions)
10 = Reserved
11 = Manager (bypass all permission checks!)

Linux uses:

Domain 0 = DOMAIN_KERNEL → Manager (0b11) – kernel bypasses permission checks
Domain 1 = DOMAIN_USER → Client (0b01) – user space checked normally
Domain 2 = DOMAIN_IO → Client (0b01)

This is a significant difference from ARM64 — ARM32 kernel runs as **domain manager**, meaning it can access any memory regardless of AP bits. ARM64 removed domains entirely.

Part 5: VA→PA Translation on ARM32 — Step by Step

Let's translate `0xC0012345` (a kernel virtual address on ARM32).

Step 0: Identify TTBR

VA = 0xC0012345

bits = 1 → kernel space

With TTBCR.N=0: use TTBR0 for everything

TTBR0 = 0x80004000 (physical address of PGD, 16 KB aligned)

Step 1: Extract Index Bits

```

VA = 0xC001_2345

bits[31:20] = 0xC00 >> ... let's compute:
0xC0012345 in binary:
1100 0000 0000 0001 0010 0011 0100 0101

bits[31:20] = 1100 0000 0000 = 0xC00 = 3072 ← L1 index
bits[19:12] = 0001 0010      = 0x12 = 18   ← L2 index
bits[11:0]  = 0011 0100 0101 = 0x345      ← page offset

```

Step 2: L1 (PGD) Lookup

```

PGD base = TTBR0 = 0x80004000

L1 index = 3072

Entry address = 0x80004000 + (3072 × 4) = 0x80004000 + 0x3000 = 0x80007000

Read 4 bytes at physical 0x80007000:

L1 entry = 0x80100002

Decode:

bits[1:0] = 0b10 → Section descriptor!

bits[31:20] = 0x801 → Physical base = 0x80100000

→ This is a 1 MB SECTION mapping (no L2 needed!)
→ Physical address = 0x80100000 | bits[19:0] of VA
→ PA = 0x80100000 | 0x12345 = 0x80112345 ✓

(Walk complete in ONE level for section mappings!)

```

Step 3 (if L1 was a Page Table, not Section): L2 Lookup

If L1 entry = 0x80200001 (bits[1:0]=01 → Coarse page table):

L2 table base = 0x80200000 (bits[31:10] of L1 entry)

L2 index = 18

Entry address = 0x80200000 + (18 × 4) = 0x80200048

Read 4 bytes at physical 0x80200048:

L2 entry = 0x90000FF2

Decode:

bits[1:0] = 0b10 → Small page (4 KB)

bits[31:12] = 0x90000 → Physical page at 0x90000000

bits[5:4] = AP = 0b11 → Full access

bit = C = 1 → Cacheable

bit = B = 1 → Bufferable (Write-Back)

Physical Address = 0x90000000 | 0x345 = 0x90000345 ✓

Full Walk Diagram (ARM32)

VA: 0xC0012345

|

| TTBR0 = 0x80004000 (PGD base)

|

▼

[PGD Table @ 0x80004000] (16 KB, 4096 entries × 4 bytes)

|

| index = bits[31:20] = 3072

| entry @ 0x80007000

▼

L1 entry = Section descriptor

|

| Physical base = bits[31:20] of entry

| + VA bits[19:0] as offset

▼

Physical Address: 0x80112345 ✓

--- OR if L1 = Page Table descriptor ---

[L2 Table @ (L1 entry bits[31:10])] (1 KB, 256 entries × 4 bytes)

|

| index = bits[19:12] = 18

| entry @ base + 18×4

▼

L2 entry = Small page descriptor

|

| Physical page = bits[31:12] of entry

| + VA bits[11:0] as offset

▼

Physical Address: 0x90000345 ✓

Part 6: ARM32 Boot — Initial Kernel Page Table Setup

This is where it gets really interesting. Here's exactly what happens from power-on to MMU enable on ARM32 Linux.

Boot Entry Point: `arch/arm/kernel/head.S`

When the kernel starts (handed off from U-Boot or bootloader), the CPU is in:

- **SVC mode** (supervisor, equivalent to EL1 on ARM64)
- **MMU OFF** — all addresses are physical
- **Caches OFF**
- PC = physical address of kernel entry (`_stext`)

```

/* arch/arm/kernel/head.S */

ENTRY(stext)

    /* CPU is in SVC mode, MMU off, caches off */

    /* r0 = 0 (must be 0 per boot protocol) */

    /* r1 = machine type ID */

    /* r2 = physical address of ATAGs/DTB */

    safe_svcmode_maskall r9    /* ensure SVC mode, IRQs/FIQs disabled */

    mrc p15, 0, r9, c0, c0    /* read MIDR (CPU ID) into r9 */

    bl __lookup_processor_type /* find CPU-specific setup */
    movs r10, r5              /* r10 = proc_info pointer */
    beq __error_p             /* unknown CPU → halt */

    bl __lookup_machine_type  /* find machine/board type */
    movs r8, r5
    beq __error_a             /* unknown machine → halt */

    ldr r13, =__mmap_switched /* address of post-MMU C entry point */

    bl __create_page_tables    /* ← THE KEY FUNCTION */

    /* Enable MMU */
    ldr r13, =__mmap_switched
    b __enable_mmu

```

`__create_page_tables` — The Heart of ARM32 Boot

This function creates the **initial page tables** in physical memory before the MMU is turned on. It uses **1 MB section mappings** for speed.

```

__create_page_tables:

    /* r4 = physical address of page table (PHYS_OFFSET - 0x4000) */
    /* The PGD is placed just below the kernel in physical memory */

    pgtbl    r4, r8                /* r4 = PGD physical address */

    /* Step 1: Zero out the entire PGD (16 KB = 4096 entries × 4 bytes) */
    mov r0, r4
    mov r3, #0
    add r6, r0, #PG_DIR_SIZE      /* PG_DIR_SIZE = 0x4000 (16 KB) */
1: str r3, [r0], #4
    str r3, [r0], #4
    str r3, [r0], #4
    str r3, [r0], #4
    teq r0, r6
    bne 1b

    /* Step 2: Load MMU flags for section mapping */
    ldr r7, [r10, #PROCINFO_MM_MMUFLAGS]
    /* r7 = PMD_TYPE_SECT | PMD_SECT_AP_WRITE | PMD_SECT_AP_READ
       | PMD_FLAGS_CACHED (TEX=001, C=1, B=1 for normal cached) */

    /* Step 3: Map the kernel itself (identity + virtual mapping) */
    /*
     * We need TWO mappings:
     * A) Identity mapping: PA=0x80000000 → VA=0x80000000
     *    (so the CPU can keep executing after MMU is turned on,
     *    since PC still has physical address at that moment)
     * B) Kernel mapping: PA=0x80000000 → VA=0xC0000000
     *    (the permanent kernel virtual address)
     */

    /* Calculate PGD index for identity mapping */
    mov r6, pc                    /* r6 = current PC (physical) */
    mov r6, r6, lsr #20           /* r6 = bits[31:20] = L1 index */

```

```

    orr r3, r7, r6, lsl #20    /* r3 = section descriptor: PA | flags */

    str r3, [r4, r6, lsl #2]   /* PGD[r6] = r3 (identity map) */

    /* Calculate PGD index for kernel virtual mapping */
    add r0, r4, #(KERNEL_START & 0xFF000000) >> 18
    /* KERNEL_START = 0xC0000000, so index = 0xC00 = 3072 */

    /* r0 = &PGD[3072] */

    ldr r6, =KERNEL_START      /* r6 = 0xC0000000 */
    ldr r5, =KERNEL_END        /* r5 = end of kernel image */
    ldr r3, =PHYS_OFFSET       /* r3 = 0x80000000 (physical RAM start) */
    orr r3, r3, r7             /* r3 = section descriptor: PHYS_OFFSET | flags */

    /* Map kernel sections: one 1MB entry per MB of kernel */
2:  str r3, [r0], #4           /* PGD[index] = section descriptor */
    add r3, r3, #1 << 20       /* next physical MB */
    cmp r6, r5                 /* done? */
    add r6, r6, #1 << 20       /* next virtual MB */
    blo 2b

    /* Step 4: Map the page tables themselves (so kernel can modify them) */
    /* Map 16KB PGD at its virtual address */
    add r0, r4, #PAGE_OFFSET >> 18
    orr r6, r4, r7             /* section descriptor for PGD physical addr */
    str r6, [r0]

    /* Step 5: Map device I/O regions (UART for early printk, etc.) */
    /* Platform-specific: maps debug UART, interrupt controller, etc. */
    /* Called via __iortable_init */

    mov pc, lr                 /* return */

```

What the PGD Looks Like After `__create_page_tables`

PGD (physical @ 0x7FFFC000, 16 KB):

Index	VA Range	Entry Type	Physical
0	0x00000000-0x000FFFFF	0 (invalid)	
...			
512	0x20000000-0x200FFFFF	0 (invalid)	
...			
2048	0x80000000-0x800FFFFF	Section → 0x80000000	← IDENTITY MAP (temporary)
2049	0x80100000-0x801FFFFF	Section → 0x80100000	
...	(covers entire kernel image physically)		
...			
3072	0xC0000000-0xC00FFFFF	Section → 0x80000000	← KERNEL VIRTUAL MAP
3073	0xC0100000-0xC01FFFFF	Section → 0x80100000	
3074	0xC0200000-0xC02FFFFF	Section → 0x80200000	
...	(maps all physical RAM to 0xC0000000+)		
...			
4095	0xFFFF0000-0xFFFFFFFF	0 (invalid)	

Part 7: `__enable_mmu` — Turning On the MMU

```
__enable_mmu:

/* Set Domain Access Control Register */

mov r5, #(domain_val(DOMAIN_USER, DOMAIN_MANAGER) | \
    domain_val(DOMAIN_KERNEL, DOMAIN_MANAGER) | \
    domain_val(DOMAIN_IO, DOMAIN_CLIENT))

mcr p15, 0, r5, c3, c0, 0 /* write DACR */

/* Write PGD physical address to TTBR0 */

mcr p15, 0, r4, c2, c0, 0 /* TTBR0 = PGD physical address */

/* Invalidate TLB (must be clean before enabling MMU) */

mcr p15, 0, r0, c8, c7, 0 /* TLBIALL – invalidate entire TLB */

/* Enable MMU: set SCTLR.M bit */

mrc p15, 0, r0, c1, c0, 0 /* read SCTLR */

orr r0, r0, #CR_M /* set M bit (MMU enable) */

orr r0, r0, #CR_C /* set C bit (D-cache enable) */

orr r0, r0, #CR_I /* set I bit (I-cache enable) */

mcr p15, 0, r0, c1, c0, 0 /* write SCTLR – MMU ON! */

/* CRITICAL: The next instruction executes with MMU ON */

/* PC still has physical address → identity map keeps us alive */

mov pc, r13 /* jump to __mmap_switched (virtual address) */
```

The Critical Moment — Identity Map Saves Us

Before MMU enable:

PC = 0x80008000 (physical)

Executing: `mcr p15, 0, r0, c1, c0, 0` (enable MMU)

After MMU enable (next instruction):

PC = 0x80008004 (still physical-looking)

MMU translates: VA 0x80008004 → identity map → PA 0x80008004 ✓

CPU keeps executing!

Then: `mov pc, r13` (r13 = 0xC0008xxx – virtual address)

PC jumps to 0xC0008xxx

MMU translates: VA 0xC0008xxx → kernel map → PA 0x80008xxx ✓

Now running at virtual kernel address!

Identity map is no longer needed after this jump.

It gets removed later in `__mmap_switched`.

Part 8: `__mmap_switched` — Post-MMU C Entry

```
__mmap_switched:
    /* Now running at virtual addresses! */

    /* Set up stack pointer */
    adr r3, __mmap_switched_data
    ldmia r3!, {r4, r5, r6, r7}
    cmp r4, r5                /* is .data already initialized? */
    ldrne fp, [r3, #-4]

    /* Clear BSS */
    mov fp, #0
1:  cmp r6, r7
    strcc fp, [r6], #4
    bcc 1b

    /* Set up initial stack */
    ldmia r3, {r4, r5, r6, sp}

    /* Store machine type and ATAGs/DTB pointer */
    str r9, [r4]              /* save CPU ID */
    str r1, [r5]              /* save machine type */
    str r2, [r6]              /* save ATAGs/DTB pointer */

    /* Jump to start_kernel() – the C entry point */
    b    start_kernel
```

Part 9: `start_kernel` → `setup_arch` → Full Page Table Setup

After `start_kernel()` is called, the kernel sets up proper page tables replacing the boot-time section mappings:

```
start_kernel()
|
|─ setup_arch()
|   |─ setup_processor()    – CPU-specific init
|   |─ setup_machine_fdt() – parse Device Tree
|   |─ arm_memblock_init() – reserve memory regions
|   └─ paging_init()       ← FULL PAGE TABLE SETUP
|       |─ build_mem_type_table() – set up TEX/CB memory types
|       |─ prepare_page_table() – clear boot mappings
|       |─ map_lowmem()       – map all lowmem with proper PTEs
|       |─ devicemaps_init() – map I/O regions
|       └─ kmap_init()       – set up PKMAP area for highmem
|
|─ mm_init()
|   └─ mem_init()           – hand pages to buddy allocator
|
└─ ... rest of kernel init
```

paging_init() — Replacing Section Maps with PTEs

```
/* arch/arm/mm/mmu.c */

void __init paging_init(const struct machine_desc *mdesc) {
    void *zero_page;

    build_mem_type_table(); /* set up memory type descriptors */
    prepare_page_table();   /* clear identity map + old boot mappings */

    /* Map all lowmem (0xC0000000 to high_memory) */
    map_lowmem();

    /* Map I/O regions from machine descriptor */
    devicemaps_init(mdesc);

    /* Set up kmap area for highmem access */
    kmap_init();

    /* Allocate zero page */
    zero_page = early_alloc(PAGE_SIZE);

    /* Switch from boot PGD to proper per-process PGD infrastructure */
    bootmem_init();
}
```

Part 10: ARM32 TLB Management

ARM32 TLB Instructions (CP15 coprocessor)

```

/* Invalidate entire TLB (all ASIDs) */
mcr p15, 0, r0, c8, c7, 0    /* TLBIALL */

/* Invalidate TLB entry by VA (all ASIDs) */
mcr p15, 0, r0, c8, c7, 1    /* TLBIMVA – r0 = VA | ASID */

/* Invalidate TLB by ASID */
mcr p15, 0, r0, c8, c7, 2    /* TLBIASID – r0 = ASID */

/* Invalidate instruction TLB */
mcr p15, 0, r0, c8, c5, 0    /* ITLBIALL */

/* Data Synchronization Barrier */
mcr p15, 0, r0, c7, c10, 4    /* DSB (pre-ARMv7 encoding) */
dsb                          /* ARMv7+ instruction */

```

ASID on ARM32

ARM32 uses **8-bit ASIDs** (256 values) stored in `CONTEXTIDR`:

```

/* Write ASID to CONTEXTIDR */
mcr p15, 0, r1, c13, c0, 1    /* CONTEXTIDR = ASID */

/* Linux ARM32 context switch: */
/* 1. Write new ASID to CONTEXTIDR */
/* 2. Write new TTBR0 */
/* 3. ISB to ensure TTBR0 is used */

```

With only 256 ASIDs (vs 65536 on ARM64), ASID rollover happens more frequently on ARM32, causing more TLB flushes.

ARM32 vs ARM64 — Side-by-Side Comparison

Aspect	ARM32 (ARMv7)	ARM64 (ARMv8/AArch64)
VA size	32-bit (4 GB)	48-bit (256 TB per half)
Page table levels	2 (PGD + PTE)	4 (PGD/PUD/PMD/PTE)
PGD size	16 KB (4096 × 4B)	4 KB (512 × 8B)
PTE size	4 bytes	8 bytes
Huge page sizes	1 MB (section), 64 KB (large page)	2 MB (L2 block), 1 GB (L1 block)
TTBR registers	TTBR0 + TTBR1 (split via TTBCR)	TTBR0_EL1 + TTBR1_EL1 (always split)
ASID bits	8-bit (256 values)	16-bit (65536 values)
Domains	Yes (DACR, 16 domains)	No (removed)
Memory types	TEX + C + B bits	AttrIndx → MAIR_EL1
Fault registers	DFSR/DFAR/IFSR/IFAR	ESR_EL1/FAR_EL1
TLB instructions	CP15 coprocessor (mcr/mrc)	TLBI system instructions
Boot page tables	1 MB section mappings	2 MB block mappings
Highmem	Yes (ZONE_HIGHMEM, kmap)	No (all RAM directly mapped)
Kernel domain	Manager (bypasses AP checks)	No domains, AP always checked
Cache maintenance	CP15 coprocessor	DC/IC system instructions

ARM32 Boot Flow Summary

Power On / U-Boot hands off

|

▼

stext (head.S) – physical addresses, MMU off

|

└─ __lookup_processor_type (identify CPU)

└─ __lookup_machine_type (identify board)

|

└─ __create_page_tables

|

└─ Zero 16 KB PGD

|

└─ Create identity map (PA=VA for kernel image)

|

└─ Create kernel virtual map (PA=0xC0000000+)

|

└─ Map I/O for early debug

|

└─ __enable_mmu

|

└─ Write DACR (domain permissions)

|

└─ Write TTBR0 (PGD physical address)

|

└─ TLBIALL (flush TLB)

|

└─ Set SCTLR.M=1 (MMU ON!)

|

└─ Identity map keeps CPU alive for 1 instruction

|

└─ Jump to __mmap_switched (virtual address)

|

└─ Clear BSS

|

└─ Set up stack

|

└─ b start_kernel()

|

└─ paging_init()

└─ Remove identity map

└─ Set up proper PTEs

└─ Map lowmem

└─ Set up highmem (PKMAP/FIXMAP)

Key Source Files for ARM32

File	Purpose
arch/arm/kernel/head.S	Boot entry, <code>__create_page_tables</code> , <code>__enable_mmu</code>
arch/arm/mm/mmu.c	<code>paging_init()</code> , <code>map_lowmem()</code> , <code>devicemaps_init()</code>
arch/arm/mm/	

SECTION 13

ARM32 Boot: Initial Kernel Page Table Setup (head.S walkthrough)

This section was covered in detail in Section 12 (ARM32 MMU translation), which included the full boot sequence: `__create_page_tables`, identity mapping, `__enable_mmu`, and `paging_init()`. The ARM32 boot flow is embedded in the ARM32 translation section above.

SECTION 14

ARM32 vs ARM64 Comparison Tables & Complete Reference

ARM32 vs ARM64 — Complete Architecture Comparison

Aspect	ARM32 (ARMv7)	ARM64 (ARMv8/AArch64)
VA size	32-bit (4 GB)	48-bit (256 TB per half)
Page table levels	2 (PGD + PTE)	4 (PGD/PUD/PMD/PTE)
PGD size	16 KB (4096 × 4B)	4 KB (512 × 8B)
PTE size	4 bytes	8 bytes
Huge page sizes	1 MB (section), 64 KB (large page)	2 MB (L2 block), 1 GB (L1 block)
TTBR registers	TTBR0 + TTBR1 (split via TTBCR)	TTBR0_EL1 + TTBR1_EL1 (always split)
ASID bits	8-bit (256 values)	16-bit (65536 values)
Domains	Yes (DACR, 16 domains)	No (removed in ARM64)
Memory types	TEX + C + B bits	AttrIndx → MAIR_EL1 (8 slots)
Fault registers	DFSR/DFAR/IFSR/IFAR	ESR_EL1/FAR_EL1/ELR_EL1
TLB instructions	CP15 coprocessor (mcr/mrc)	TLBI system instructions
Boot page tables	1 MB section mappings	2 MB block mappings
Highmem	Yes (ZONE_HIGHMEM, kmap/kunmap)	No (all RAM directly mapped)
Kernel domain	Manager (bypasses AP checks)	No domains, AP always checked
Cache maintenance	CP15 coprocessor	DC/IC system instructions
Context switch	Write CONTEXTIDR (8-bit ASID)	Write TTBR0_EL1 with 16-bit ASID

ARM64 System Register Reference

Register	Purpose
TTBR0_EL1	User space page table base + ASID
TTBR1_EL1	Kernel space page table base
TCR_EL1	Translation control: VA size, page size, cache attrs
MAIR_EL1	Memory attribute index array (8 memory types)
ESR_EL1	Exception Syndrome Register (fault type + DFSC level)
FAR_EL1	Fault Address Register (virtual address that faulted)
ELR_EL1	Exception Link Register (faulting instruction address)
SCTLR_EL1	System Control Register (MMU enable, cache enable)
CPACR_EL1	Architectural Feature Access Control
CTR_EL0	Cache Type Register (cache line size)

ARM32 System Register Reference

Register	Purpose
TTBR0	User/process page table base
TTBR1	Kernel page table base (when TTBCR.N > 0)
TTBCR	Translation Table Base Control (VA split config)
DACR	Domain Access Control Register (16 domains × 2 bits)
SCTLR	System Control Register (MMU enable = bit M)
DFSR/DFAR	Data Fault Status/Address Registers
IFSR/IFAR	Instruction Fault Status/Address Registers
CONTEXTIDR	Context ID Register (contains 8-bit ASID)
MIDR	Main ID Register (CPU identification)

ARM64 PTE Field Reference (4KB Granule)

Field	Bits	Description
Valid	[1:0]=11	Page descriptor (valid entry)
Valid (block)	[1:0]=01	Block descriptor (huge page)
Output Address	[47:12]	Physical page base address
AF	bit 10	Access Flag (set on first access, used for LRU)
SH	[9:8]	Shareability (11=Inner Shareable)
AP	[7:6]	Access Permissions (00=RW kernel, 01=RO user)
NS	bit 5	Non-Secure bit
AttrIndx	[4:2]	Index into MAIR_EL1 (memory type)
UXN	bit 54	User Execute Never (EL0 cannot execute)
PXN	bit 53	Privileged Execute Never (EL1 cannot execute)
DBM	bit 51	Dirty Bit Modifier (hardware dirty tracking)
Contiguous	bit 52	Hint: entry is part of contiguous TLB range
nG	bit 11	Not Global (ASID-tagged TLB entry if 1)

ARM64 ESR_EL1 DFSC Values (Data Fault Status Code)

DFSC Value	Fault Type
0b000100	Translation fault, L0
0b000101	Translation fault, L1
0b000110	Translation fault, L2
0b000111	Translation fault, L3 (PTE not present — most common)
0b001001	Access flag fault, L1
0b001010	Access flag fault, L2
0b001011	Access flag fault, L3 (AF=0, first access)
0b001101	Permission fault, L1
0b001110	Permission fault, L2
0b001111	Permission fault, L3 (write to read-only — CoW!)

ARM64 MAIR_EL1 Memory Types (Linux Default)

Index	Type	Cache Behavior	Use Case
0	MT_DEVICE_nGnRnE	Strongly ordered, no gather/reorder	MMIO registers
1	MT_DEVICE_nGnRE	Device, no gather/reorder	PCIe MMIO
2	MT_DEVICE_GRE	Device, gather+reorder allowed	Write-combining
3	MT_NORMAL_NC	Non-cacheable normal	Coherent DMA buffers
4	MT_NORMAL	Write-Back, Write-Allocate (fully cached)	Normal RAM
5	MT_NORMAL_WT	Write-Through, no Write-Allocate	Rarely used

Key Linux Kernel Source Files Reference

ARM64 Memory Management

File	Purpose
arch/arm64/mm/mmu.c	Kernel page table setup, map_kernel_range()
arch/arm64/mm/fault.c	do_mem_abort(), do_page_fault()
arch/arm64/mm/init.c	paging_init(), map_mem()
arch/arm64/mm/pgd.c	PGD allocation for user processes
arch/arm64/mm/dma-mapping.c	DMA mapping implementation
arch/arm64/include/asm/pgtable.h	PTE manipulation macros
arch/arm64/include/asm/pgtable-hwdef.h	PTE hardware bit definitions
arch/arm64/include/asm/tlbflush.h	TLB maintenance operations

ARM32 Memory Management

File	Purpose
arch/arm/kernel/head.S	Boot entry, __create_page_tables, __enable_mmu
arch/arm/mm/mmu.c	paging_init(), map_lowmem(), devicemaps_init()
arch/arm/mm/fault.c	do_page_fault(), do_bad_area()
arch/arm/mm/highmem.c	kmap()/kunmap() implementation
arch/arm/include/asm/pgtable.h	ARM32 PTE macros
arch/arm/include/asm/domain.h	Domain Access Control definitions

Generic Memory Management

File	Purpose
mm/page_alloc.c	Buddy allocator: __alloc_pages(), __free_pages(), expand()
mm/slub.c	SLUB allocator implementation
mm/compaction.c	Memory compaction: compact_zone(), kcompactd
mm/cma.c	Contiguous Memory Allocator: cma_alloc(), cma_release()
mm/vmalloc.c	vmalloc() implementation
mm/dma-contiguous.c	DMA CMA integration
kernel/dma/mapping.c	Generic DMA mapping API
drivers/iommu/arm/arm-smmu-v3/	ARM SMMUv3 driver
include/linux/mmzone.h	struct zone, struct free_area, zone types
include/linux/slab.h	kmalloc/kmem_cache public API

Allocator Quick Reference

ALLOCATION DECISION TREE:

Need kernel memory?

|

└─ DMA / hardware?

| └─ Small repeated (descriptors) → dma_pool_alloc()

| └─ Large buffer → dma_alloc_coherent()

|

└─ Physically contiguous (non-DMA)?

| └─ Small (<~4MB) → kmalloc() / kcalloc()

| └─ Multiple pages → alloc_pages(GFP_KERNEL, order)

|

└─ Physical contiguity not needed?

| └─ Large buffer → vmalloc() / vzalloc()

|

└─ Frequent fixed-size objects?

| └─ kmem_cache_create() + kmem_cache_alloc()

|

└─ Per-CPU data?

| └─ alloc_percpu() / DEFINE_PER_CPU()

|

└─ Must never fail (I/O path)?

 └─ mempool_alloc()

GFP Flag Selection Guide

CONTEXT	CORRECT GFP FLAG
Process context, can sleep	→ GFP_KERNEL
Interrupt handler	→ GFP_ATOMIC
Spinlock held	→ GFP_ATOMIC
I/O path (avoid deadlock)	→ GFP_NOIO
Filesystem path	→ GFP_NOFS
DMA buffer needed	→ GFP_KERNEL GFP_DMA
Zero-initialized	→ GFP_KERNEL __GFP_ZERO (or use kzalloc())
Critical, must not fail	→ GFP_KERNEL __GFP_NOFAIL

Debugging Tools Reference

Tool / File	Purpose
/proc/buddyinfo	Buddy allocator free lists per zone per order
/proc/pagetypeinfo	Free lists per migration type
/proc/slabinfo	SLUB/slab cache statistics (name, objects, size)
/proc/vmallocinfo	vmalloc allocations with caller info
/proc/meminfo	System-wide memory statistics including CMA
/sys/kernel/slab/<name>/	Per-cache SLUB statistics and controls
/sys/kernel/debug/cma/	CMA region statistics
slabtop	Real-time slab cache memory usage
CONFIG_KASAN	Kernel Address Sanitizer (use-after-free, OOB)
CONFIG_KMEMLEAK	Memory leak detection
CONFIG_DEBUG_PAGEALLOC	Page allocation debugging
CONFIG_DMA_API_DEBUG	DMA mapping error detection